Stack, Queue and Circular Queue

built from fixed-size singly linked lists

Function-by-function guide to **dsa.h** and **dsa.cpp**

For each function this guide gives you
What the function does, and what it receives as parameters
Why each parameter is there, and whether it is passed by value or by reference
The process step by step, with a picture of the nodes while it runs
The real code from the final dsa.cpp
For queue and circular queue: only what is different from the stack, and why
The menus that connect everything, a comparison table, and likely viva questions

Settings in the code: maxsize = 6 nodes per list, up to 100 lists of each kind. All overflow, underflow and wrap checks described here are exactly the ones in the final files.

Contents

1. How to read this guide **4**

2. Building blocks **5**

2.1 The node and the fixed list . . . 5

2.2 Which pointers describe each structure . . . 5

2.3 The func struct (dsa.h) . . . 6

3. The two helper functions: createList and prevNode **7**

3.1 createList . . . 7

3.2 prevNode . . . 8

4. Insert and remove: push / qpush / cpush and pop / qpop / cpop **9**

4.1 Insert . . . 9

4.2 Remove . . . 13

5. Showing and searching: display, count, search **17**

5.1 Stack: display and search . . . 17

5.2 Queue: qdisplay, qcount, qsearch . . . 18

5.3 Circular queue: cdisplay, ccount, csearch . . . 19

6. Reverse and remove-from-rear (queue and circular queue) **21**

6.1 qreverse and creverse . . . 21

6.2 qpopLast and cpopLast . . . 22

7. The question functions (Q1 to Q10) **23**

7.1 Q1: split into two (splitStack, splitQueue, splitCircular) . . . 23

7.2 Q2: even and odd (evenOdd, qevenOdd, cevenOdd) . . . 28

7.3 Q3: next greater element (nextGreater, qnextGreater, cnextGreater) . . . 30

7.4 Q5: sort (sortList, qsortList, csortList) . . . 32

7.5 Q6: palindrome (isPalindrome, qisPalindrome, cisPalindrome) . . . 34

7.6 Q7: copy (copyStack, copyQueue, copyCircular) . . . 35

7.7 Q8: balanced brackets (isBalanced, qisBalanced, cisBalanced) . . . 36

7.8 Q9: remove duplicates (removeDuplicates, qremoveDuplicates, cremoveDuplicates) . . . 38

7.9 Q10: intersection (intersection, qintersection, cintersection) . . . 40

8. The menus: how the functions are connected **42**

8.1 Start-up: main and menu . . . 42

8.2 Working on the selected list 42
8.3 Create, select and show all 43
8.4 Menu choices that create new lists (Q1, Q2, Q7, Q10) 43
8.5 Q10 in the menu: four checks 44
8.6 Word questions (Q6 and Q8) 45

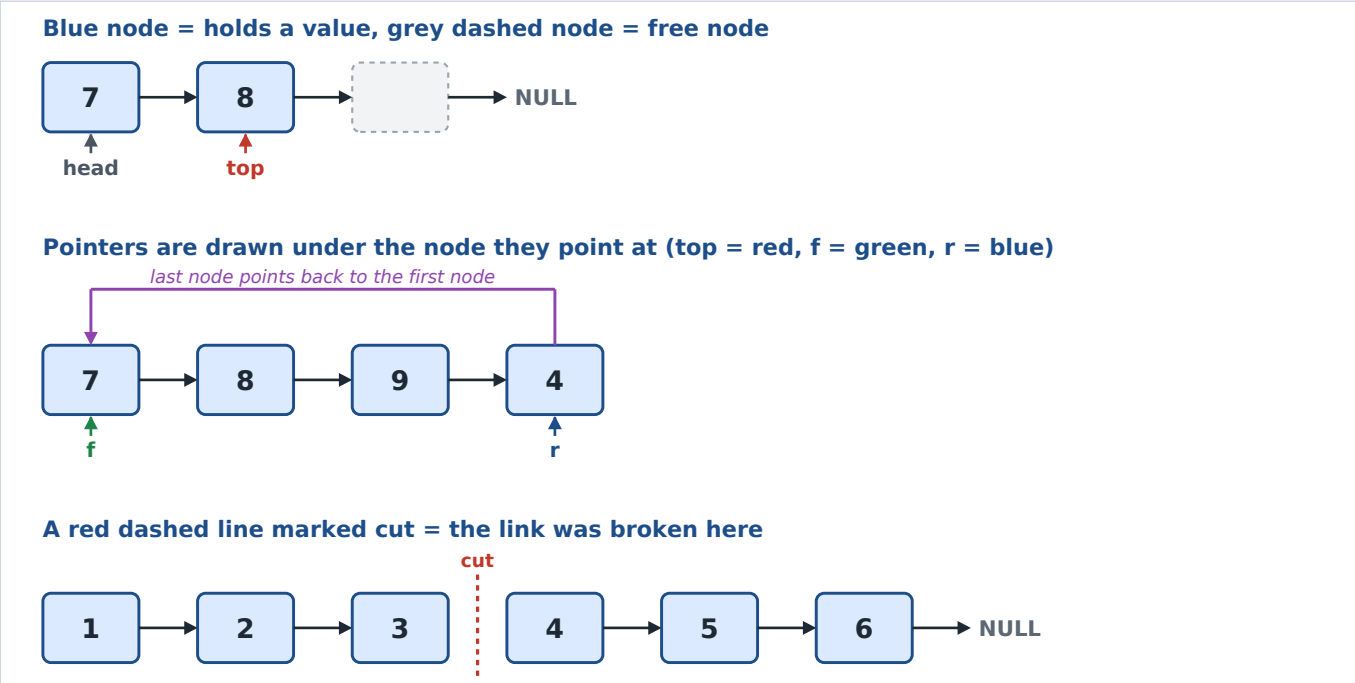
9. Comparison of the three structures **46**

10. Likely viva questions **47**

1. How to read this guide

This guide explains every function in **dsa.h** and **dsa.cpp**: what it receives, why each parameter is there, what it does step by step, and a picture of the nodes while it runs. The three data structures are always shown in the same order: **Stack, then Queue, then Circular Queue**. The stack version is explained in full. For the queue and the circular queue the guide only explains **what is different** (different parameters, different process) and **why**.

Picture legend



A purple arc over the nodes means a ring: the last node points back to the first node.

What 'Passed how' means in the parameter tables

- **by value** (`node *head`): the function gets a copy of the pointer. It can read and change the *data inside the nodes* (the nodes are shared) but it cannot move the caller's pointer.
- **by reference** (`node *&top`): the function gets the caller's own pointer. When the function moves it (for example *top = top->next*) the caller sees the move. Any pointer that a function must move or set is passed by reference.

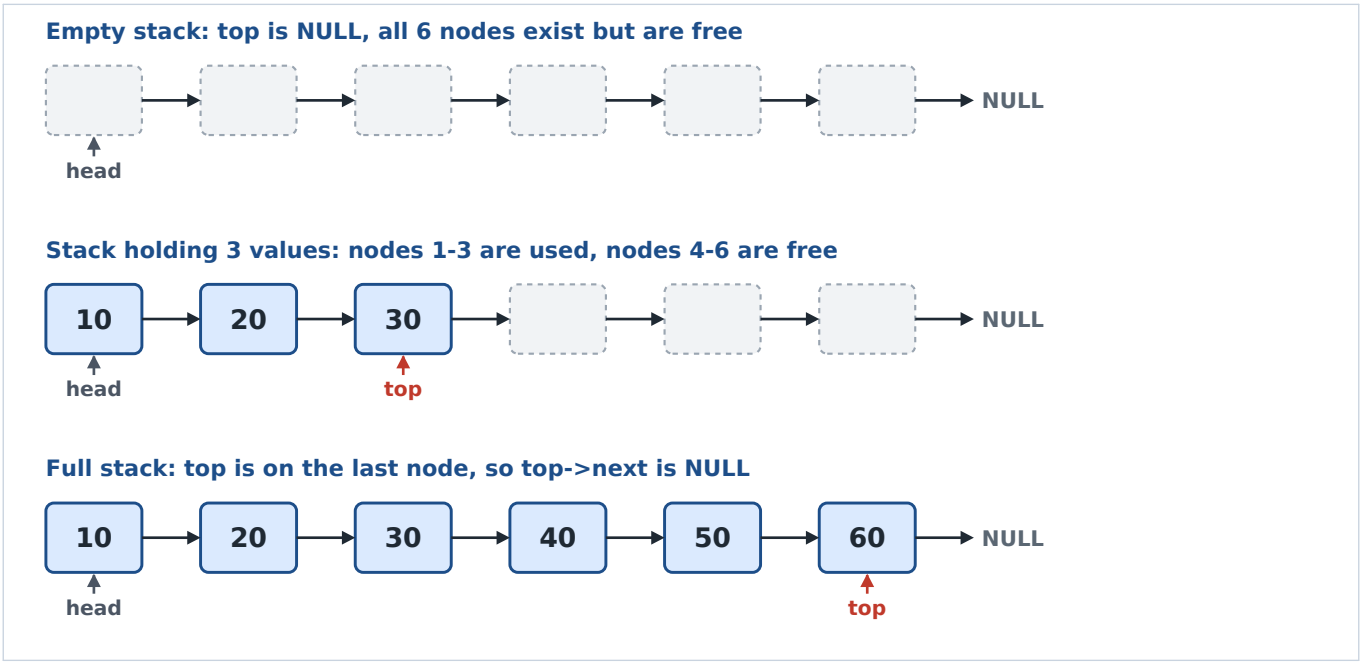
2. Building blocks

2.1 The node and the fixed list

Everything is built from one struct: `struct node { int data; node *next; }`. A node holds one value and points to the next node. A list is only a chain of nodes, so the program never stores a list in an array.

The lists here are **fixed size**. **createList** builds all **6 nodes** (the value of *maxsize*) before anything is inserted. After that, no node is created or deleted while the program runs. Insert and remove only **move pointers**. This gives two different numbers:

- **capacity** = number of nodes = 6. This never changes.
- **size** = number of nodes that hold a value right now = anything from 0 to 6.



Overflow is detected by running out of nodes, not by comparing a counter with a number.

2.2 Which pointers describe each structure

	Stack	Queue	Circular Queue
Pointers kept	**head** and **top**	**head**, **f** (front) and **r** (rear)	**head**, **f** and **r**
Empty when	top == NULL	f == NULL (and r == NULL)	f == NULL and r == NULL
Used nodes	from head up to top	from f up to r	from f forward to r (may wrap past the last node)
Last node's next	NULL	NULL	head (a ring)
Insert at	top (the same end as remove)	rear	rear
Remove from	top	front	front

A stack has **one** working end, the top. A queue has **two**, front and rear. That one difference explains most of the differences in the code that follows: a stack needs only one moving pointer, a queue needs two, and because the nodes only point forward the stack has to search from the head whenever it needs to step

backwards from the top.

2.3 The func struct (dsa.h)

All functions are members of one struct, **func**, which also stores every list. This is what lets the menu handle many stacks, queues and circular queues with the same functions.

Member	Meaning
maxsize = 6	Number of nodes in every list. Change it here and every list changes size.
MAXS = 100	Most lists of each kind that can exist.
shead[i], stop[i]	Stack number i: its head and its top. (Arrays of 100 pointers.)
qhead[i], qfront[i], qrear[i]	Queue number i: its head, front and rear.
chead[i], cfront[i], crear[i]	Circular queue number i: its head, front and rear.
snum, qnum, cnum	How many stacks / queues / circular queues exist now.
scur, qcur, ccur	Index of the list that the menu is working on (the selected list).
stmp, qtmp, ctmp	One temporary list of each kind. Used by palindrome, brackets and remove-duplicates so the user's own lists are not touched.

User-facing numbers start at 1 but arrays start at 0, so the menus convert with *a--* or *n - 1*.

> **Why arrays of lists and not 'stack1, stack2'**
> If the code had variables named stack1 and stack2 it would need a new function for each. With arrays, **one** push works for list number 1, list number 2 or list number 100. The menu only changes the number of the selected list (scur, qcur or ccur), and the functions never know which list they work on.

3. The two helper functions: createList and prevNode

These are the only helper functions in the program. Every other function depends on them, so they are explained first.

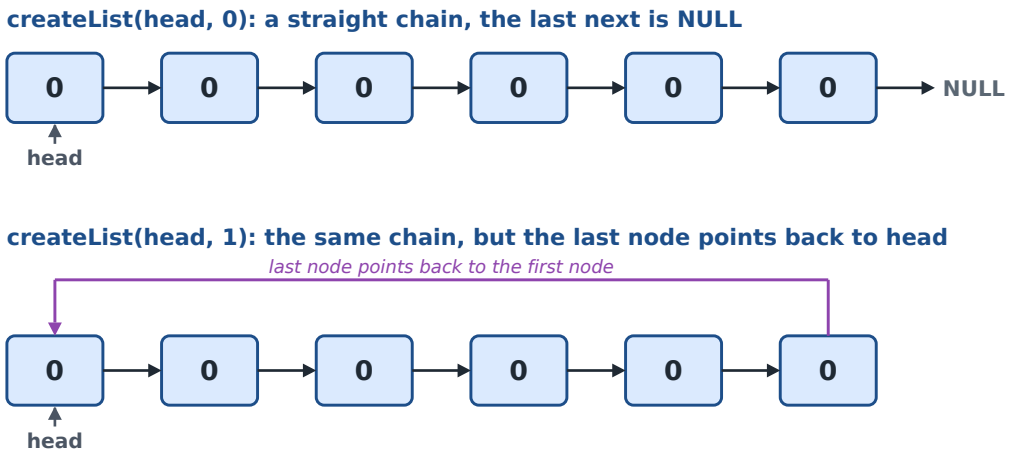
3.1 createList

What it does: builds a chain of *maxsize* (6) nodes, all with data 0. If it is a circular queue, the last node is joined back to the first node.

Parameter	Passed how	Why the function needs it
head	by reference (*node *&*)	The function creates the first node and must hand it to the caller. The caller passes *shead[i]*, *qhead[i]* or *chead[i]* and that array slot becomes the new first node.
circular	by value (int)	0 for stack and queue (the last node's next is NULL), 1 for circular queue (the last node's next is head). One function builds both kinds.

Step by step

1. Create the first node with *new*, set its data to 0, and let *head* point to it.
2. Repeat 5 more times: create a node, set data 0, link it after the previous node with *p->next = n*.
3. After the loop, *p* is the last node, so set *p->next = NULL*.
4. If *circular == 1*, change that last link to *p->next = head*.



The 0 values are placeholders. Nodes that are free are shown grey in later pictures.

> **Why the last node points to head in a circular queue**
> With that one link the pointer *r = r->next* or *f = f->next* automatically moves from the last node to the first node. That is the 'wrap around' and no special code is needed to move around the circle.

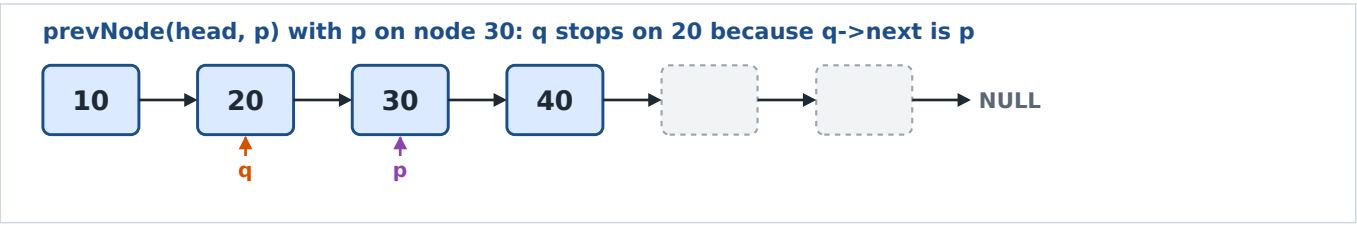
```
void func::createList(node *&head, int circular)
{
    head = new node;
    head->data = 0;
    node *p = head;
    for (int i = 1; i < maxsize; i++)
    {
        node *n = new node;
        n->data = 0;
        p->next = n;
        p = n;
    }
    p->next = NULL;
    if (circular == 1)
        p->next = head;
}
```

3.2 prevNode

What it does: returns the node that comes *before* node *p*, searching forward from *start*. Returns NULL if *p* is the start node, because nothing comes before it.

Parameter	Passed how	Why the function needs it
start	by value	Where to begin the search: the head of a stack, or the front of a queue. The search must begin at the first used node.
p	by value	The node whose predecessor we want.

Why it exists: the nodes have only a *next* pointer, so a node cannot tell which node points to it. To step backwards, the only way is to start at the beginning and walk forward until *q->next == p*.



q walks 10, then 20. At 20, q->next == p, so the loop stops and 20 is returned.

- **Stack** uses it in pop, display and nextGreater, because the top must move *down* the stack.
- **Queue and circular queue** use it in reverse and popLast, where the rear must move backwards.
- Cost: it walks from the start each time, so it is slow for long lists. This is the price of a singly linked list.

```
node *func::prevNode(node *start, node *p)
{
    if (p == start)
        return NULL;
    node *q = start;
    while (q->next != p)
        q = q->next;
    return q;
}
```

4. Insert and remove: push / qpush / cpush and pop / qpop / cpop

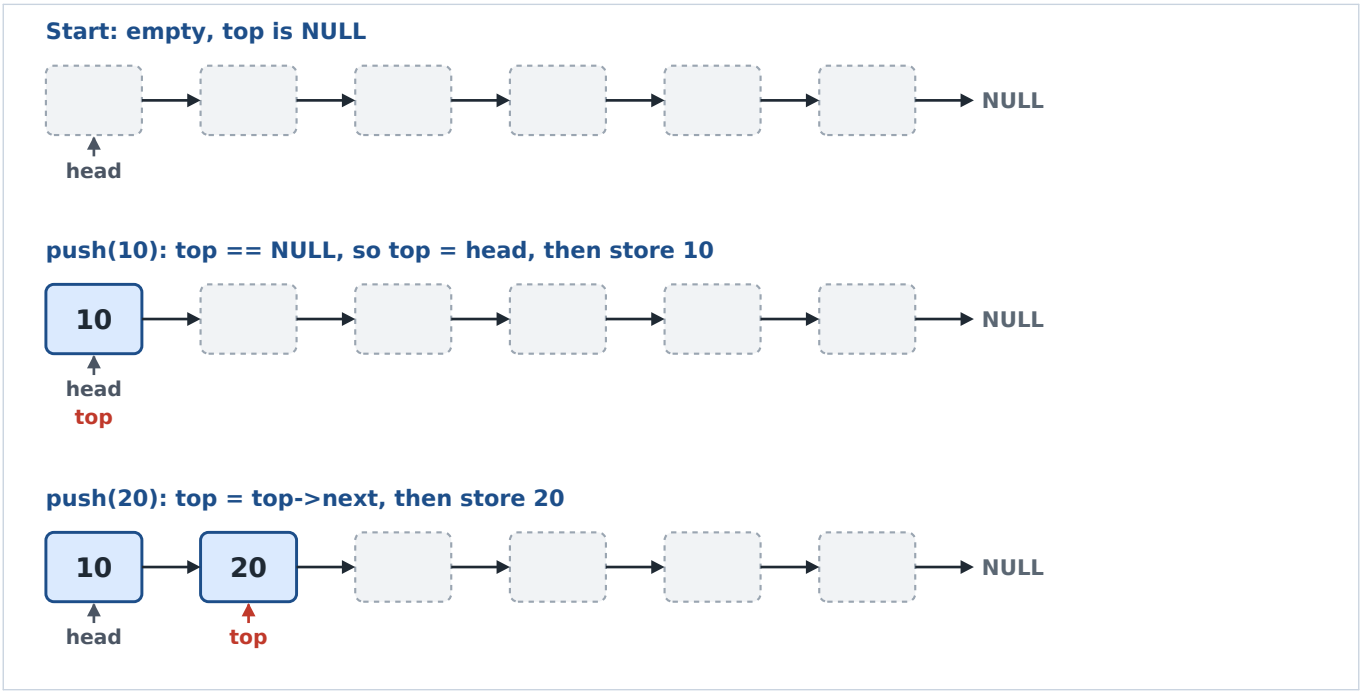
4.1 Insert

Stack: push

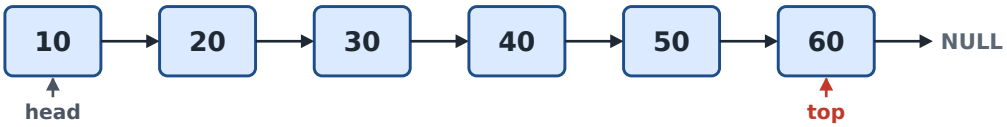
What it does: puts a new value on top of the stack. If every node is already used, it prints **Stack Overflow**.

Parameter	Passed how	Why the function needs it
head	by value	Needed only when the stack is empty: the first value goes into the head node, and top starts there.
top	by reference (*node *&*)	The function moves top up by one node. The caller must see the new top, so it is passed by reference.
x	by value (int)	The value to store.

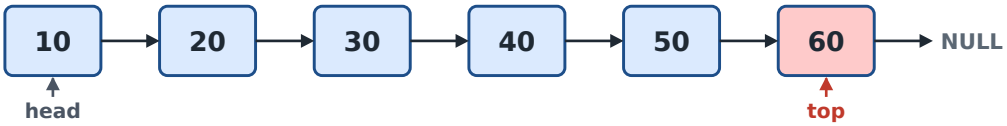
1. **top == NULL** (stack is empty): set *top* = *head*, store x in it, return.
2. **top->next == NULL** (top is on the last node, so there is no free node above it): print **Stack Overflow**, return.
3. Otherwise move *top* = *top->next* and store x in that node.



After 6 pushes: top is on the last node



push(70): top->next == NULL, so it prints Stack Overflow and nothing changes



push never creates a node. It only moves top onto the next free node that already exists.

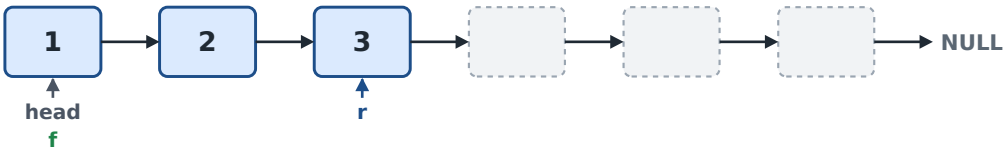
```
void func::push(node *head, node *&top, int x)
{
    if (top == NULL)
    {
        top = head;
        top->data = x;
        return;
    }
    if (top->next == NULL)
    {
        cout << "Stack Overflow" << endl;
        return;
    }
    top = top->next;
    top->data = x;
}
```

Queue: qpush

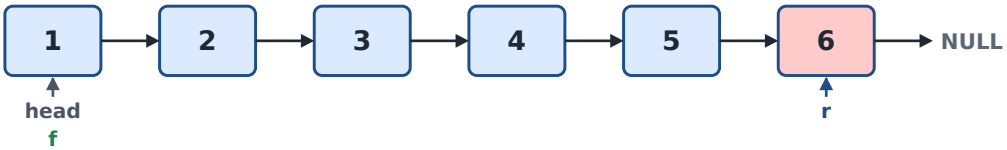
Difference from the stack

Parameters: *f* and *r* are both passed by reference instead of one *top*. The first insert has to set **both** ends (*f = head; r = head*), and later inserts move *r*. *head* is still by value.
Empty check: *r == NULL* (the stack used *top == NULL*). It means the same: nothing is in the queue.
Process: the move happens at the **rear** (*r = r->next*). The front does not move on an insert.
Overflow check: *r->next == NULL*. Because qpop recycles nodes (see 4.2), this is true only when all 6 nodes really hold values.

Queue with 3 values: f on the first node, r on the third



Full: r->next == NULL, so qpush prints Queue Overflow



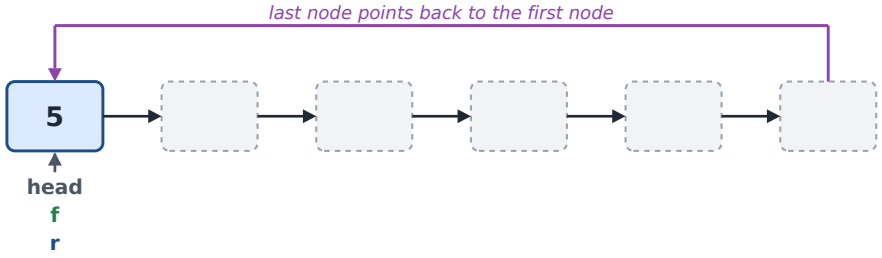
```
void func::qpush(node *head, node *&f, node *&r, int x)
{
    if (r == NULL)
    {
        f = head;
        r = head;
        r->data = x;
        return;
    }
    if (r->next == NULL)
    {
        cout << "Queue Overflow" << endl;
        return;
    }
    r = r->next;
    r->data = x;
}
```

Circular queue: cpush (the four checks)

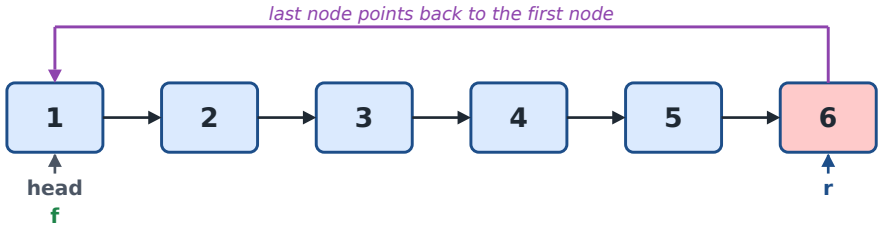
Difference from the stack
Parameters: the same as qpush (*head* by value, *f* and *r* by reference). *head* is needed to detect the ring's ends and to wrap.
Process: there is no NULL end, so 'is there room?' cannot be answered by *r->next == NULL*. Four ordered checks replace it. They run **in this order** and the first one that fits is the one that acts.

Check	Condition in code	Meaning	Action
1	f == NULL && r == NULL	Queue is empty	f = head, r = head, store x
2	f == head && r->next == head	Front is on the first node and rear is on the last node: nothing has wrapped yet, so it is a full **simple** queue	print Queue Overflow
3	r->next == f	Front is exactly one node after rear: the circle is full after wrapping	print Circular Queue Overflow
4	r->next == head (then wrap), otherwise move	If rear is on the last node it goes back to the first node (wrap). Otherwise it moves one node forward	move r, store x

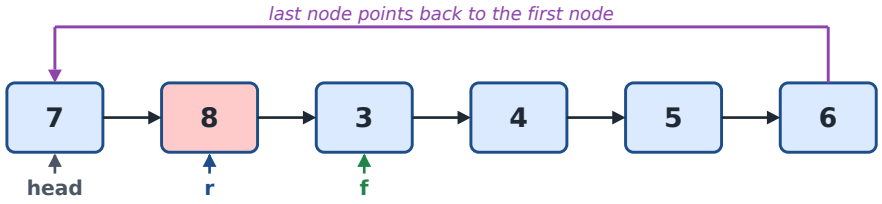
Check 1 happened: first insert, f and r both on the head node



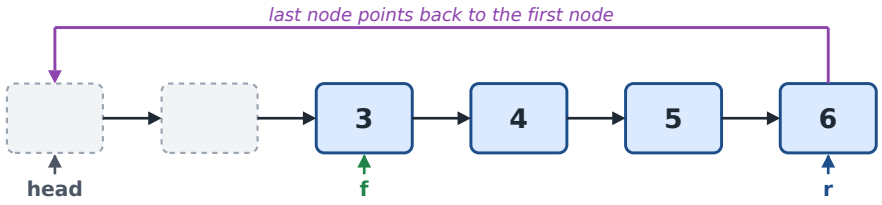
Check 2: f == head and r->next == head. Full as a simple queue. Message: Queue Overflow



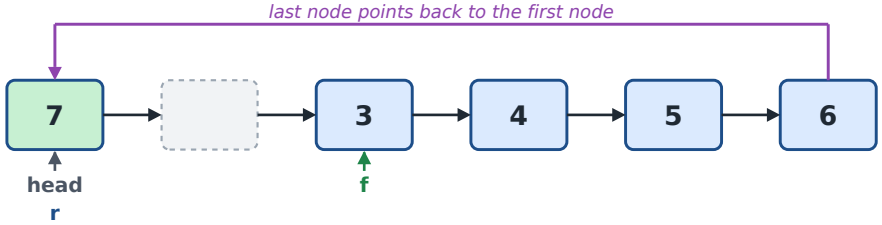
Check 3: r->next == f. Rear has wrapped and caught up with front. Message: Circular Queue Overflow



Before check 4: rear is on the last node and there is room at the start



Check 4 wrapped r from the last node to head, then stored 7



Check 2 is really a special case of check 3 (when f is the head, r->next == head means r->next == f). It is placed first so that a queue that never wrapped reports the simple-queue message.

Why the order matters

Check 1 must be first, because r->next would crash if r were NULL. Checks 2 and 3 must come before check 4, because check 4 changes r: the full test has to happen before the move. This is also why a circular queue can act as a simple queue: until rear wraps, only checks 1, 2 and 4 are used.

```
void func::cpush(node *head, node *&f, node *&r, int x)
{
    // Check 1: front and rear are both NULL (queue empty),
    // so both go to the first node
    if (f == NULL && r == NULL)
    {
        f = head;
        r = head;
        r->data = x;
        return;
    }

    // Check 2 (simple queue): front is on the first node and
    // rear is on the last node, so the queue is full
    if (f == head && r->next == head)
    {
        cout << "Queue Overflow" << endl;
        return;
    }

    // Check 3 (circular queue): front is one node after rear
    // (front = rear + 1), so the circle is full
    if (r->next == f)
    {
        cout << "Circular Queue Overflow" << endl;
        return;
    }

    // Check 4: rear is on the last node, so it goes back to the first node.
    // Otherwise rear simply moves one node forward.
    if (r->next == head)
        r = head;
    else
        r = r->next;

    r->data = x;
}
```

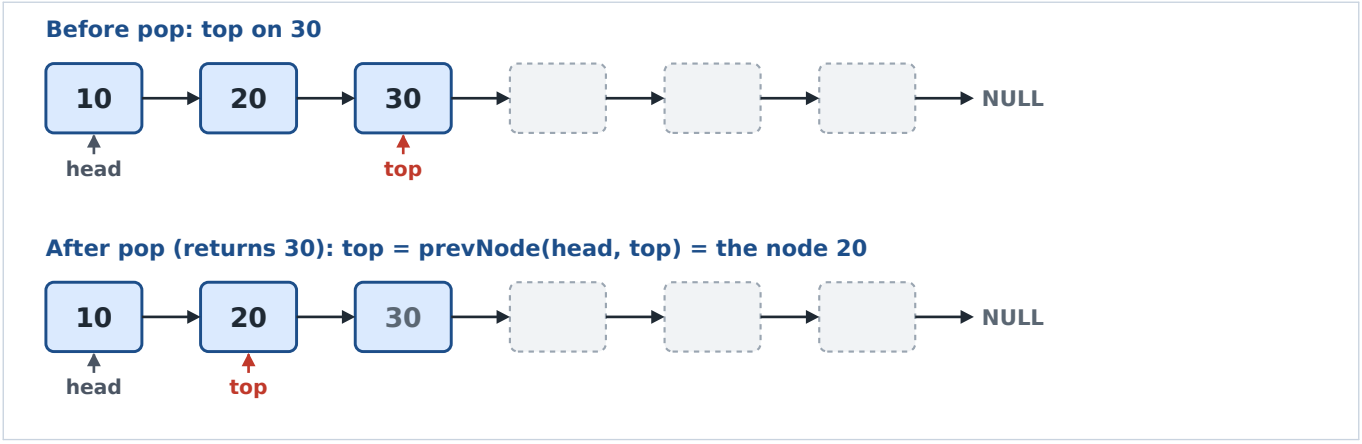
4.2 Remove

Stack: pop

What it does: removes the top value and returns it. On an empty stack it prints **Stack Underflow** and returns -1.

Parameter	Passed how	Why the function needs it
head	by value	pop must move top *down* one node. With only *next* pointers the only way is prevNode(head, top), which searches from head.
top	by reference	The function moves top. The caller must see the new top.

1. **top == NULL**: print **Stack Underflow**, return -1.
2. Save the value of the top node in x.
3. Move *top = prevNode(head, top)*. If the top was the head, prevNode returns NULL, so top becomes NULL and the stack is empty.
4. Return x.



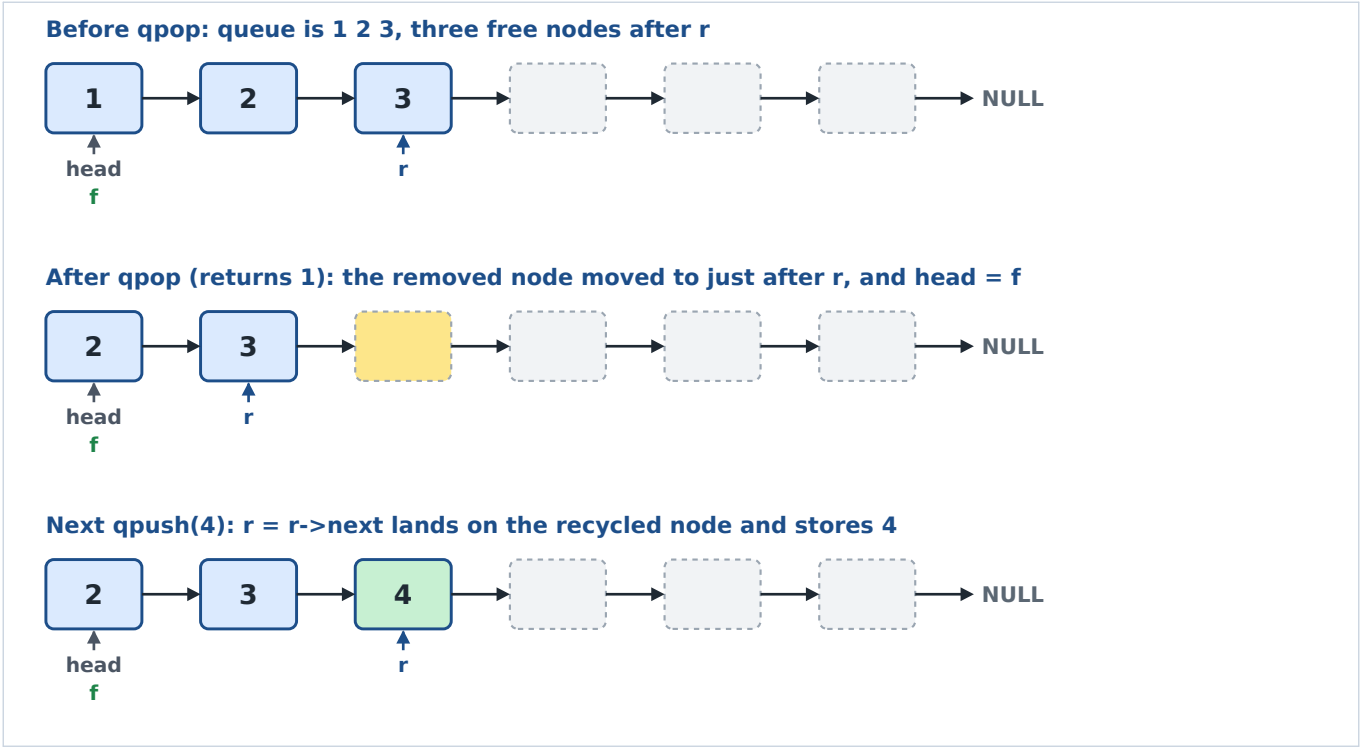
The 30 is still in the node, but the node is now above top, so it counts as free and the next push overwrites it.

```
int func::pop(node *head, node *&top)
{
    if (top == NULL)
    {
        cout << "Stack Underflow" << endl;
        return -1;
    }
    int x = top->data;
    top = prevNode(head, top);
    return x;
}
```

Queue: qpop

> **Difference from the stack**
> **Parameters:** *head* is passed **by reference** here (pop for a stack had it by value). qpop changes the first node of the whole chain, so the caller's head must change too. There is no prevNode because the front simply moves **forward**: *f = f->next*.
> **Process:** after the front moves on, the node just removed is **recycled**. It is cut out of the front and linked in again right after the rear. Without this, a plain queue would never reuse the space at its front.

1. **f == NULL**: print **Queue Underflow**, return -1.
2. Save the front value in x.
3. **f == r** (only one value): set f and r to NULL and return x. The chain is unchanged and the next insert starts at head.
4. Otherwise: *e = f* (the node being removed); *f = f->next*; *head = f* (the chain now starts at the new front); *e->next = r->next* (e points to whatever followed the rear: the free nodes or NULL); *r->next = e* (e now follows the rear).
5. Return x.



The yellow node is the old first node. It was recycled, so the queue still has 6 nodes in total and Queue Overflow only happens when all 6 hold values.

```
int func::qpop(node *&head, node *&f, node *&r)
{
    if (f == NULL)
    {
        cout << "Queue Underflow" << endl;
        return -1;
    }
    int x = f->data;
    if (f == r)
    {
        f = NULL;
        r = NULL;
        return x;
    }
    node *e = f;
    f = f->next;
    head = f;
    e->next = r->next;
    r->next = e;
    return x;
}
```

Circular queue: cpop

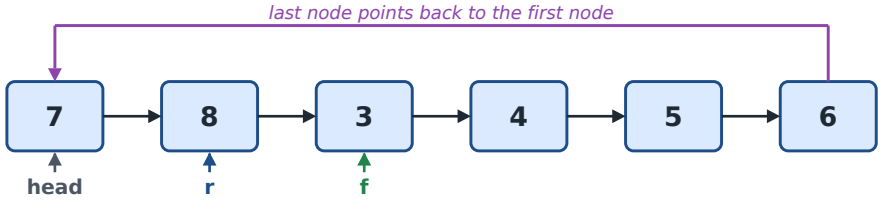
Difference from the stack

Parameters: only *f* and *r*, both by reference. **No head** is needed and there is no rotation, because the ring already reuses space by wrapping.

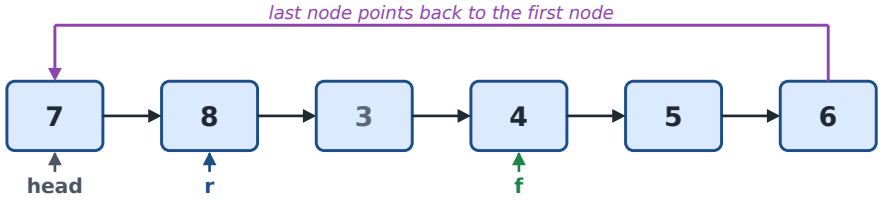
Process: *f = f->next*. If front was on the last node, its next is head, so the front **wraps to the first node by itself**.

Checks: (1) empty gives Circular Queue Underflow. (2) *f == r* means one value, so both pointers become NULL and the next insert starts at the first node again (the same rule as check 1 of cpush).

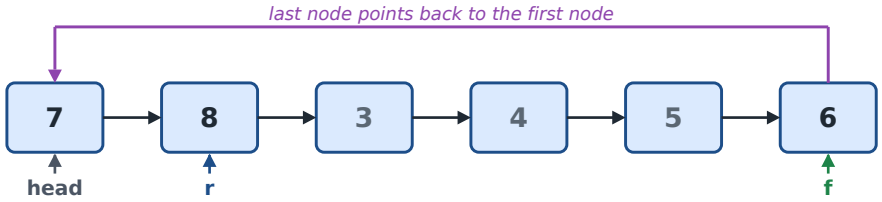
Before: f on 3, wrapped queue 3 4 5 6 7 8



After cpop (returns 3): f = f->next, no other change



Later, f on the last node: cpop makes f = f->next, which is head




```
int func::cpop(node *&f, node *&r)
{
    // Check 1: queue is empty
    if (f == NULL)
    {
        cout << "Circular Queue Underflow" << endl;
        return -1;
    }

    int x = f->data;

    // Check 2: only one element left, so the queue becomes empty.
    // Both pointers go to NULL and the next insert starts at the first node.
    if (f == r)
    {
        f = NULL;
        r = NULL;
        return x;
    }

    // Move front forward. If front is on the last node,
    // its next is the first node, so front wraps to the first node.
    f = f->next;
    return x;
}
```

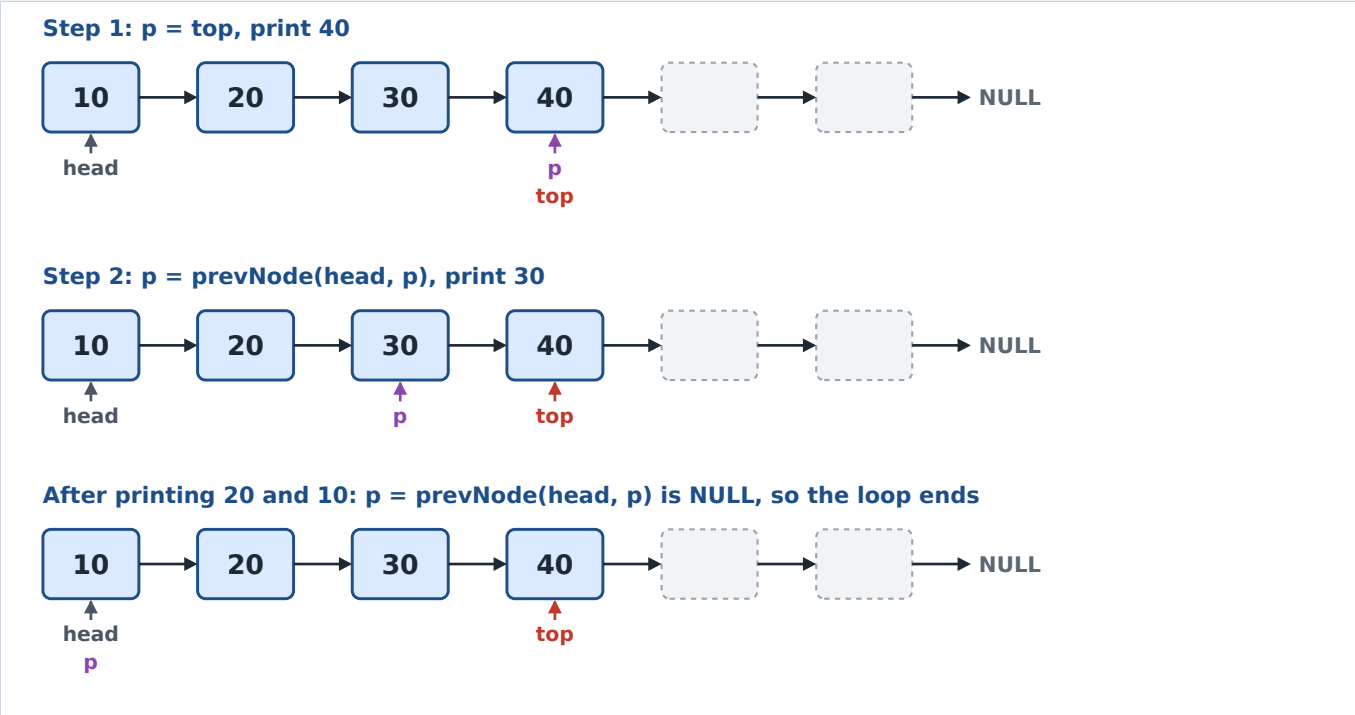
5. Showing and searching: display, count, search

5.1 Stack: display and search

display

What it does: prints the stack from the **top** down to the bottom, so the value pushed last is printed first. An empty stack prints *empty*.

Parameter	Passed how	Why the function needs it
head	by value	Needed for prevNode: the printing walks *down* from the top, so each step asks prevNode(head, p) for the node below.
top	by value	Where the printing starts. It is only read, never changed, so no reference is needed.



Output: 40 30 20 10

```
void func::display(node *head, node *top)
{
    if (top == NULL)
    {
        cout << "empty" << endl;
        return;
    }
    node *p = top;
    while (p != NULL)
    {
        cout << p->data << " ";
        p = prevNode(head, p);
    }
    cout << endl;
}
```

search

What it does: returns 1 if x is found among the stack's *used* nodes, otherwise 0. It is used by Q9 and Q10.

Parameter	Passed how	Why the function needs it
head, top	by value	The search goes *forward* from head to top. The used nodes are exactly the ones between them.
x	by value	The value to look for.

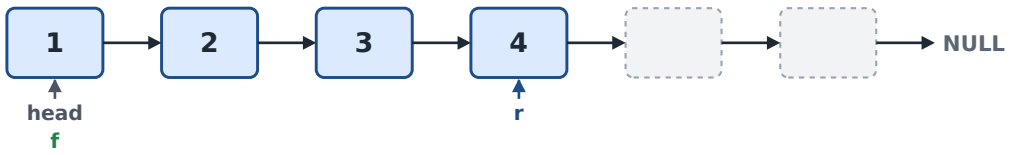
The loop stops at **end = top->next**, the first free node. Without this the loop would also read the free nodes above top, which still hold old values and would give wrong answers. An empty stack returns 0 at once.

```
int func::search(node *head, node *top, int x)
{
    if (top == NULL)
        return 0;
    node *p = head;
    node *end = top->next;
    while (p != end)
    {
        if (p->data == x)
            return 1;
        p = p->next;
    }
    return 0;
}
```

5.2 Queue: qdisplay, qcount, qsearch

Difference from the stack
Parameters: only *f* and *r*. There is **no head**, because the walk goes **forward** from the front to the rear and never needs to step back. That is why a queue needs no prevNode here.
Process: the walk starts at *f* and stops at *end = r->next*. Output is in the order of insertion: oldest value first. The stack has no counting function. **qcount** exists because the queue menu has a Count choice and because the split, reverse and sort functions need n.

qdisplay: p goes from f forward and stops at r->next (the first free node). Output 1 2 3 4




```
void func::qdisplay(node *f, node *r)
{
    if (f == NULL)
    {
        cout << "empty" << endl;
        return;
    }
    node *p = f;
    node *end = r->next;
    while (p != end)
    {
        cout << p->data << " ";
        p = p->next;
    }
    cout << endl;
}
```

```
int func::qcount(node *f, node *r)
{
    if (f == NULL)
        return 0;
    int c = 0;
    node *p = f;
    node *end = r->next;
    while (p != end)
    {
        c++;
        p = p->next;
    }
    return c;
}

int func::qsearch(node *f, node *r, int x)
{
    if (f == NULL)
        return 0;
    node *p = f;
    node *end = r->next;
    while (p != end)
    {
        if (p->data == x)
            return 1;
        p = p->next;
    }
    return 0;
}
```

5.3 Circular queue: cdisplay, ccount, csearch

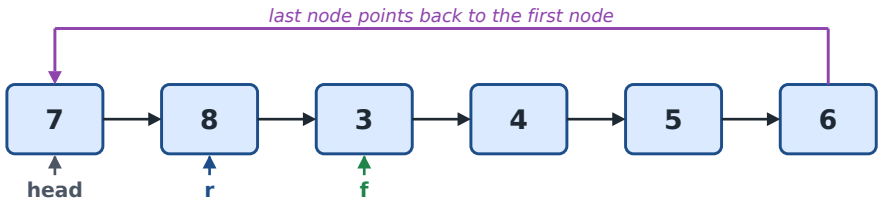
Difference from the stack

Parameters: the same as the queue (*f* and *r* only).

Process: the walk still goes from *f* to *end* = *r->next*, but it is written as a **do ... while** instead of a while loop. When the circle is full, *r->next* == *f*, so *end* equals the starting node. A normal while loop would see *p* == *end* at once and print nothing. do...while runs once first, then tests.

The loop passes through the last node and continues at the first node by itself (the wrap), until it reaches rear.

Full wrapped circle: end = r->next = f. Printing starts at f and passes the last node into the first. Output 3 4 5 6 7 8



cdisplay, ccount and csearch all use this same do...while shape.

```
void func::cdisplay(node *f, node *r)
{
    if (f == NULL)
    {
        cout << "empty" << endl;
        return;
    }
    node *p = f;
    node *end = r->next;    // node after rear (front itself when the circle is full)
    do                      // do...while so a full circle is still printed once
    {
        cout << p->data << " ";
        p = p->next;
    } while (p != end);
    cout << endl;
}
```

```
int func::ccount(node *f, node *r)
{
    if (f == NULL)
        return 0;
    int c = 0;
    node *p = f;
    node *end = r->next;
    do
    {
        c++;
        p = p->next;
    } while (p != end);
    return c;
}

int func::csearch(node *f, node *r, int x)
{
    if (f == NULL)
        return 0;
    node *p = f;
    node *end = r->next;
    do
    {
        if (p->data == x)
            return 1;
        p = p->next;
    } while (p != end);
    return 0;
}
```

6. Reverse and remove-from-rear (queue and circular queue)

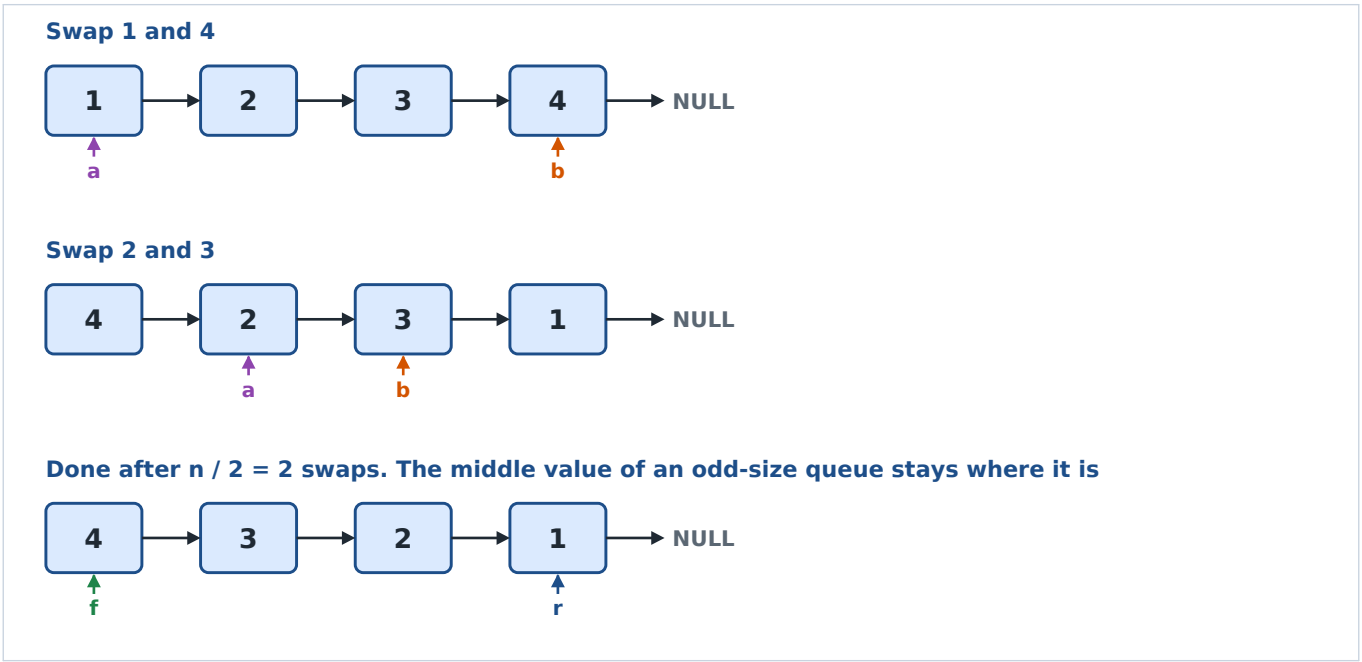
These four functions exist only for the queue and the circular queue. A stack has no reverse in this program, and it does not need remove-from-rear because its pop already removes from the working end.

6.1 qreverse and creverse

What it does: reverses the order of the values in the queue by swapping **the values** (not the nodes) from both ends towards the middle.

Parameter	Passed how	Why the function needs it
f	by value	Where the forward pointer *a* starts.
r	by value	Where the backward pointer *b* starts. The front and rear themselves do not move, only the data inside the nodes, so no reference is needed.

1. n = qcount(f, r). We need *n / 2* swaps.
2. a = f (moves forward with *a = a->next*), b = r (moves backward with *b = prevNode(f, b)*).
3. Swap a->data and b->data, move both, repeat n/2 times.




```
void func::qreverse(node *f, node *r)
{
    int n = qcount(f, r);
    node *a = f;
    node *b = r;
    for (int i = 0; i < n / 2; i++)
    {
        int t = a->data;
        a->data = b->data;
        b->data = t;
        a = a->next;
        b = prevNode(f, b);
    }
}
```

Circular queue difference
creverse is identical, except n comes from ccount. prevNode(f, b) also works in a ring: it walks from f forward until it finds the node whose next is b, and b never reaches f inside the loop, so the ring causes no problem.

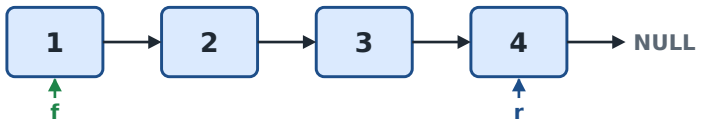
6.2 qpopLast and cpopLast

What it does: removes and returns the value at the **rear**. A normal queue cannot do this. It is used by the bracket check (Q8) so that a queue can act like a stack: the bracket to match is always the *newest* one, which sits at the rear.

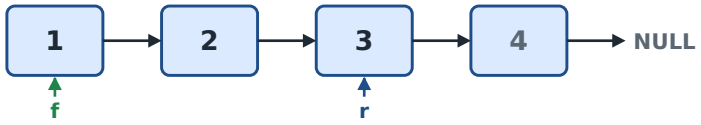
Parameter	Passed how	Why the function needs it
f	by reference	If the queue had one value, it becomes empty, so f must be set to NULL.
r	by reference	r moves back one node (*r = prevNode(f, r)*). The caller must see this.

1. **f == NULL**: print Underflow, return -1.
2. Save the value at the rear: x = r->data.
3. **f == r**: one value, so f and r become NULL.
4. Otherwise r = prevNode(f, r) (the node before the rear). Return x.

Before qpopLast



After qpopLast (returns 4): rear moved back




```
int func::qpopLast(node *&f, node *&r)
{
    if (f == NULL)
    {
        cout << "Queue Underflow" << endl;
        return -1;
    }
    int x = r->data;
    if (f == r)
    {
        f = NULL;
        r = NULL;
    }
    else
        r = prevNode(f, r);
    return x;
}
```

Circular queue difference
cpopLast prints 'Circular Queue Underflow' and has the same two checks and the same move. The only difference is the message text.

7. The question functions (Q1 to Q10)

Every question function works on lists **that are passed in**. The functions never decide *which* list to use. The menu picks the list and passes its pointers. Results that are new lists are written into the next free slot of the arrays so the user can select them and keep working.

7.1 Q1: split into two (splitStack, splitQueue, splitCircular)

What it does: makes two new lists out of one. The first holds the first half of the nodes and the second holds the rest. With 6 nodes, each half has **3 nodes**, so each half overflows after 3 values. The original list is never changed.

How: copy the base list into a new list, then **cut that copy** in the middle by changing one link. No smaller list is built and no node is created for the halves.

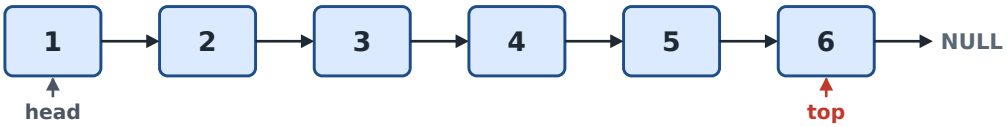
Stack: splitStack

Parameter	Passed how	Why the function needs it
head	by value	First node of the base stack. Used to count the nodes (m) and to copy from.
top	by value	Top of the base stack. Tells us how many values (n) there are and where to stop copying.
h1, t1	by reference	**Output:** head and top of the first half. The caller stores them in *shead[a]* and *stop[a]*.
h2, t2	by reference	**Output:** head and top of the second half. The caller stores them in *shead[b]* and *stop[b]*.
return value	int	1 = split done. 0 = the stack has fewer than 2 nodes, so it cannot be split (the menu prints a message).

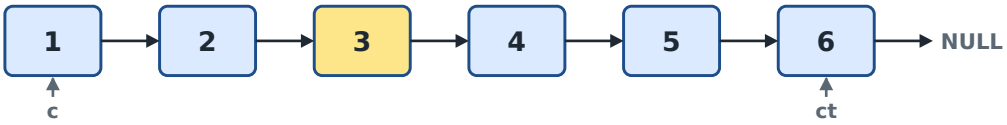
Step by step (the example has 6 values: 1 2 3 4 5 6)

1. **m** = number of nodes in the base stack (walk head to NULL). **n** = number of values (walk head to top). Here m = 6, n = 6.
2. Build a new list *c* with createList and copy the base stack into it with copyStack (which uses push). *ct* is the copy's top.
3. Keep exactly m nodes in the copy: go to the m-th node and set its next to NULL. (With m equal to maxsize this changes nothing. It guards a base list of a different size.)
4. **k = (m + 1) / 2** = 3 nodes for the first half. Find *mid*, the k-th node.
5. **The cut:** *h1 = c; h2 = mid->next; mid->next = NULL;*
6. Set the tops: if all values fit in the first half (*n <= k*) then t1 = ct and the second stack is empty (t2 = NULL). Otherwise t1 = mid (the first half is full) and t2 = ct.
7. **Reverse the values of the second half** so that stack 2's top sits next to the cut: the first value of that half (4) becomes the top, the last value (6) goes to the bottom.

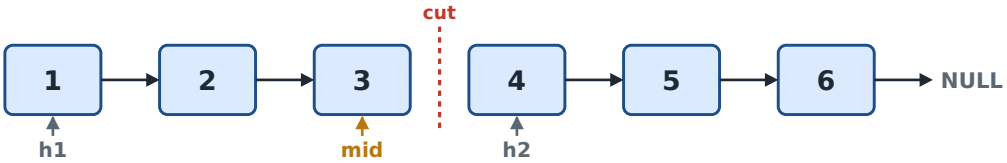
Base stack (stays unchanged): m = 6, n = 6



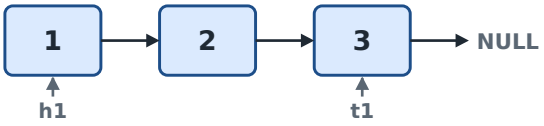
Step 2: the copy c. Step 4: k = 3, so mid is the 3rd node



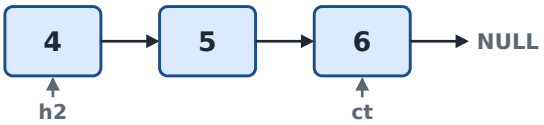
Step 5: mid->next = NULL cuts the chain. h1 is the first node, h2 is the node after mid



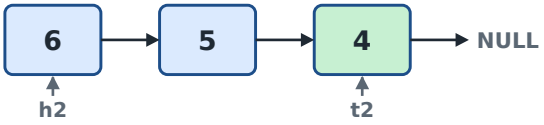
Stack 1 (first half), top t1 = mid. It overflows after 3 values



Stack 2 before the reversal: the top t2 = ct is on the last node



Step 7: values reversed. Bottom is 6, top is 4



Display prints from the top: stack 1 shows 3 2 1 and stack 2 shows 4 5 6. Both tops sit at the cut (3 and 4).

When there are fewer values than nodes

4 values: n = 4 > k = 3, so stack 1 is full (3 2 1) and stack 2 holds one value (4) with two free nodes above it.

2 values: n = 2 <= k = 3, so everything stays in stack 1 (t1 = ct) and stack 2 is empty (t2 = NULL).

```
int func::splitStack(node *head, node *top, node *&h1, node *&t1, node *&h2, node *&t2)
{
    // m = nodes in the base stack, n = values in it
    int m = 0;
    for (node *p = head; p != NULL; p = p->next)
        m++;
    if (m < 2)
        return 0;
    int n = 0;
    if (top != NULL)
        for (node *p = head; p != top->next; p = p->next)
```

```
            n++;

    // copy the base stack into a new full-size list
    node *c = NULL, *ct = NULL;
    createList(c, 0);
    copyStack(head, top, c, ct);

    // keep only m nodes, so the copy has the same size as the base stack
    node *last = c;
    for (int i = 1; i < m; i++)
        last = last->next;
    last->next = NULL;

    // cut after the k-th node: the first half ends there, the second half starts after it
    int k = (m + 1) / 2;
    node *mid = c;
    for (int i = 1; i < k; i++)
        mid = mid->next;
    h1 = c;
    h2 = mid->next;
    mid->next = NULL;

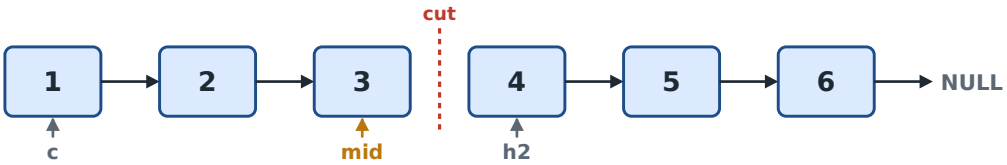
    // tops: all values fit in the first half, or they spill into the second
    if (n <= k)
    {
        t1 = ct;
        t2 = NULL;
    }
    else
    {
        t1 = mid;
        t2 = ct;

        // the top of stack 2 must be on its first value (the one next to the cut),
        // so reverse the values of the second half: the last value goes to the bottom
        int cnt = n - k;                        // values in the second half
        for (int i = 0; i < cnt / 2; i++)
        {
            node *a = h2, *b = h2;
            for (int j = 0; j < i; j++)
                a = a->next;                    // i-th node from the bottom
            for (int j = 0; j < cnt - 1 - i; j++)
                b = b->next;                    // i-th node from the top
            int tmp = a->data;
            a->data = b->data;
            b->data = tmp;
        }
    }
    return 1;
}
```

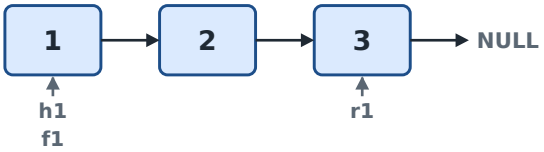
Queue: splitQueue, what is different

Parameters: the queue has three pointers per list instead of two. So the base list is (*head, f, r*) and each half is (*h, f, r*). That is why the signature has 9 parameters instead of 6. The outputs are still by reference.
Counting: n comes from **qcount(f, r)** (front to rear). The stack counted with its own loop.
Copy: copyQueue replaces copyStack (it inserts with qpush from the front).
Pointers of the halves: *f1 = cf* (the copy's front is the first node). If everything fits (*n <= k*) then r1 = cr and the second queue is empty. Otherwise r1 = mid, f2 = h2 and r2 = cr.
No reversal: the front of a queue is its oldest value. The natural order is already correct, so the halves read **1 2 3** and **4 5 6**.

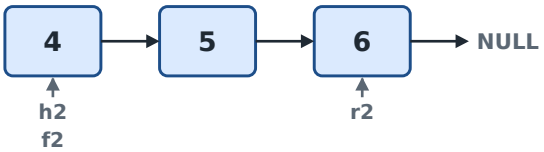
Same cut as the stack: mid->next = NULL



Queue 1: f1 = copy's front, r1 = mid. Shows 1 2 3



Queue 2: f2 = h2, r2 = cr. Shows 4 5 6




```
int func::splitQueue(node *head, node *f, node *r, node *&h1, node *&f1, node *&r1, node *&h2, node *&f2, node *&r2)
{
    // m = nodes in the base queue, n = values in it
    int m = 0;
    for (node *p = head; p != NULL; p = p->next)
        m++;
    if (m < 2)
        return 0;
    int n = qcount(f, r);

    // copy the base queue into a new full-size list
    node *c = NULL, *cf = NULL, *cr = NULL;
    createList(c, 0);
    copyQueue(f, r, c, cf, cr);

    // keep only m nodes, so the copy has the same size as the base queue
    node *last = c;
    for (int i = 1; i < m; i++)
        last = last->next;
    last->next = NULL;

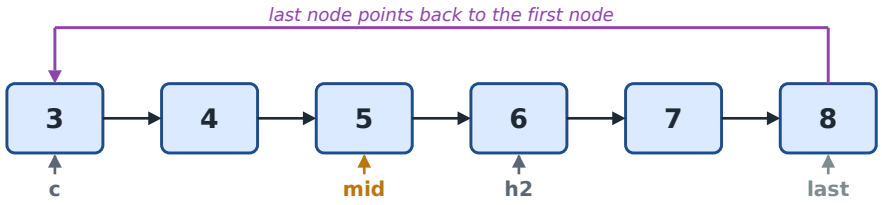
    // cut after the k-th node: the first half ends there, the second half starts after it
    int k = (m + 1) / 2;
    node *mid = c;
    for (int i = 1; i < k; i++)
        mid = mid->next;
    h1 = c;
    h2 = mid->next;
    mid->next = NULL;

    // front and rear: all values fit in the first half, or they spill into the second
    f1 = cf;
    if (n <= k)
    {
        r1 = cr;
        f2 = NULL;
        r2 = NULL;
    }
    else
    {
        r1 = mid;
        f2 = h2;
        r2 = cr;
    }
    return 1;
```

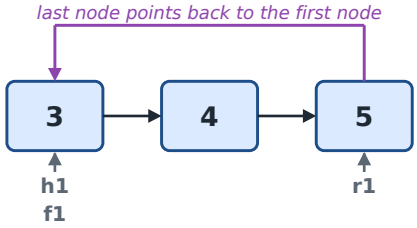
}

Circular queue: splitCircular, what is different
Parameters: the same 9 as the queue.
Counting m: a ring has no NULL end, so m is counted by going round until we are back at head (a do...while).
The cut: the ring must be closed **twice**. The code first finds *last* (the m-th node) **and then** *mid* (the k-th node), **before** changing any link. Then *mid->next = c* (first ring: its last node points to its first node) and *last->next = h2* (second ring: its last node points to its first node).
Order matters: if mid->next were changed first, the walk to find *last* would never reach it correctly.
Copy and count: copyCircular (uses cpush) and ccount. The front/rear logic at the end is the same as the queue.

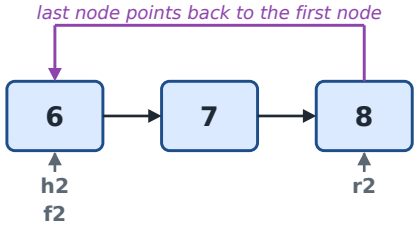
Copy of a wrapped circular queue (values from front to rear: 3 4 5 6 7 8)



Ring 1: mid->next = c



Ring 2: last->next = h2



Each half overflows after 3 values. A full half shows the Check 2 message 'Queue Overflow' until its rear wraps.

```
int func::splitCircular(node *head, node *f, node *r, node *&h1, node *&f1, node *&r1, node *&h2, node *&f2, node *&r2)
{
    // m = nodes in the base ring (go round until we are back at head), n = values in it
    int m = 0;
    node *q = head;
    do
    {
        m++;
        q = q->next;
    } while (q != head);
    if (m < 2)
        return 0;
    int n = ccount(f, r);

    // copy the base queue into a new full-size ring
    node *c = NULL, *cf = NULL, *cr = NULL;
    createList(c, 1);
    copyCircular(f, r, c, cf, cr);

    // last = the m-th node of the copy, the ring will end there
```

```
    node *last = c;
    for (int i = 1; i < m; i++)
        last = last->next;

    // mid = the k-th node, the first half ends there
    int k = (m + 1) / 2;
    node *mid = c;
    for (int i = 1; i < k; i++)
        mid = mid->next;

    h1 = c;
    h2 = mid->next;
    mid->next = c;          // first ring: its last node points back to its first node
    last->next = h2;        // second ring: its last node points back to its first node

    // front and rear: all values fit in the first half, or they spill into the second
    f1 = cf;
    if (n <= k)
    {
        r1 = cr;
        f2 = NULL;
        r2 = NULL;
    }
    else
    {
        r1 = mid;
        f2 = h2;
        r2 = cr;
    }
    return 1;
}
```

7.2 Q2: even and odd (evenOdd, qevenOdd, cevenOdd)

What it does: goes through the list once. Every even value is inserted into one new list and every odd value into another. The original is not changed. The menu creates the two new lists first, so they appear as real lists you can select later.

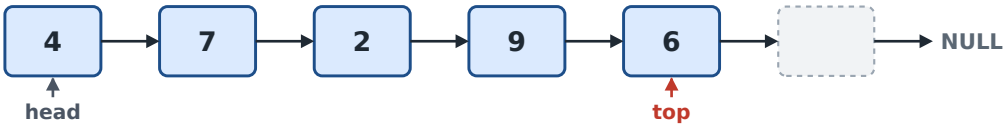
Stack: evenOdd

Parameter	Passed how	Why the function needs it
head, top	by value	The source stack. The walk goes from head up to top. It is only read.
eh, et	eh by value, et by reference	The even stack. *eh* is the head of the new list (push needs it for the first value). *et* is its top: the function sets it, so it is a reference.
oh, ot	oh by value, ot by reference	The same two things for the odd stack.

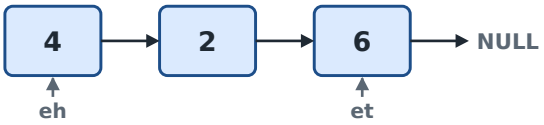
1. Set *et* and *ot* to NULL (both new stacks start empty).
2. If the source is empty, return.
3. Walk *p* from head to *end* = *top->next*.
4. If *p->data % 2 == 0* call **push(eh, et, value)** else **push(oh, ot, value)**.

It always calls the **base push**, so the overflow check is the same as everywhere else. The walk goes bottom to top so each new stack keeps the original order.

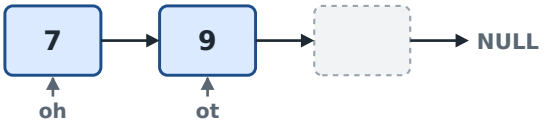
Source stack (bottom 4, top 6). The menu shows it as 6 9 2 7 4



Even stack: shows 6 2 4



Odd stack: shows 9 7




```
void func::evenOdd(node *head, node *top, node *eh, node *&et, node *oh, node *&ot)
{
    et = NULL;
    ot = NULL;
    if (top == NULL)
        return;
    node *p = head;
    node *end = top->next;
    while (p != end)
    {
        if (p->data % 2 == 0)
            push(eh, et, p->data);
        else
            push(oh, ot, p->data);
        p = p->next;
    }
}
```

Queue: qevenOdd, what is different

Parameters: queues have two pointers per list, so the result lists are passed as (*eh, evF, evR*) and (*oh, odF, odR*) and the source as (*f, r*) with no head, because the walk only goes forward.

Process: the same loop but from *f* to *r->next*, calling **qpush** for each value.

```
void func::qevenOdd(node *f, node *r, node *eh, node *&evF, node *&evR, node *oh, node *&odF, node *&odR)
{
    evF = NULL;
    evR = NULL;
    odF = NULL;
    odR = NULL;
    if (f == NULL)
        return;
    node *p = f;
    node *end = r->next;
    while (p != end)
    {
        if (p->data % 2 == 0)
            qpush(eh, evF, evR, p->data);
        else
            qpush(oh, odF, odR, p->data);
        p = p->next;
    }
}
```

Circular queue: cevenOdd, what is different
Parameters: the same as the queue.
Process: the loop is a **do...while** (full circle), and it calls **cpush** so the circular overflow checks apply to the new lists too.

```
void func::cevenOdd(node *f, node *r, node *eh, node *&evF, node *&evR, node *oh, node *&odF, node *&odR)
{
    evF = NULL;
    evR = NULL;
    odF = NULL;
    odR = NULL;
    if (f == NULL)
        return;
    node *p = f;
    node *end = r->next;
    do
    {
        if (p->data % 2 == 0)
            cpush(eh, evF, evR, p->data);
        else
            cpush(oh, odF, odR, p->data);
        p = p->next;
    } while (p != end);
}
```

7.3 Q3: next greater element (nextGreater, qnextGreater, cnextGreater)

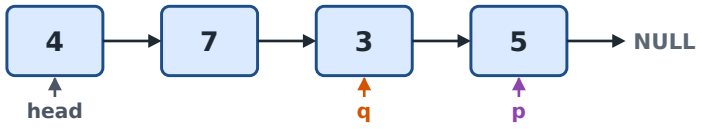
What it does: for every value, prints the first value after it that is greater than it, or **-1** if there is none. 'After it' means the direction in which values would *leave* the list.

Stack: nextGreater

Parameter	Passed how	Why the function needs it
head	by value	Needed by prevNode, because 'after' in a stack means **below**: the order in which values would be popped.
top	by value	The outer loop starts at the top. Nothing is changed, so no reference.

1. Outer loop: p goes from the top down to the bottom (*p = prevNode(head, p)*).
2. Inner loop: q starts at the node below p and also goes down.
3. The first q with *q->data > p->data* is the answer (stop with break). If q runs off the bottom (NULL) the answer stays -1.
4. Print *p->data -> answer*.

Stack 4 7 3 5 (top = 5). p = 5: q looks at 3 (no), then 7 (yes)



Output (menu shows the stack as 5 3 7 4): 5 -> 7, 3 -> 7, 7 -> -1, 4 -> -1

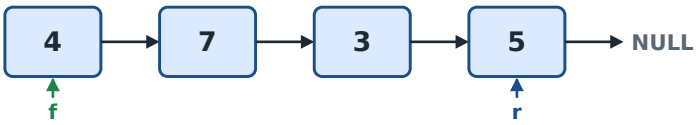
```
void func::nextGreater(node *head, node *top)
{
    if (top == NULL)
    {
        cout << "Stack is empty" << endl;
        return;
    }
    node *p = top;
    while (p != NULL)
    {
        int nge = -1;
        node *q = prevNode(head, p);
        while (q != NULL)
        {
            if (q->data > p->data)
            {
                nge = q->data;
                break;
            }
            q = prevNode(head, q);
        }
        cout << p->data << " -> " << nge << endl;
        p = prevNode(head, p);
    }
}
```

Queue: qnextGreater, what is different

Parameters: only *f* and *r*. The search for 'after' goes **forward** (toward the rear), so there is no prevNode and no head.

Process: p goes from f to *r->next*. q starts at *p->next* and goes forward to the same end. A queue has a real end, so a value near the rear may get -1.

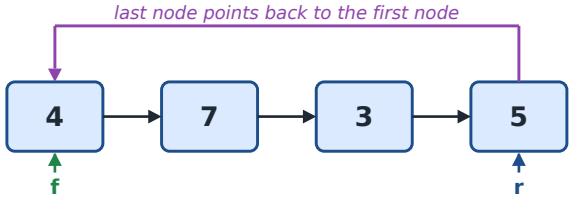
Queue 4 7 3 5. Output: 4 -> 7, 7 -> -1, 3 -> 5, 5 -> -1




```
void func::qnextGreater(node *f, node *r)
{
    if (f == NULL)
    {
        cout << "Queue is empty" << endl;
        return;
    }
    node *end = r->next;
    node *p = f;
    while (p != end)
    {
        int nge = -1;
        node *q = p->next;
        while (q != end)
        {
            if (q->data > p->data)
            {
                nge = q->data;
                break;
            }
            q = q->next;
        }
        cout << p->data << " -> " << nge << endl;
        p = p->next;
    }
}
```

Circular queue: cnextGreater, what is different
Parameters: the same as the queue.
Process: the search **wraps**. After the rear it continues from the front, and it looks at the other n - 1 values. There is no NULL end, so the loops are counted: *n = ccount(f, r)*, and the inner loop runs n - 1 times. Inside, *if (q == r) q = f; else q = q->next;* is the wrap.
Result: 5 now finds 7 (after the rear it wraps to the front: 4 is not greater, 7 is).

Circular 4 7 3 5. Output: 4 -> 7, 7 -> -1, 3 -> 5, 5 -> 7 (found after wrapping)




```
void func::cnextGreater(node *f, node *r)
{
    if (f == NULL)
    {
        cout << "Circular Queue is empty" << endl;
        return;
    }
    int n = ccount(f, r);
    node *p = f;
    for (int i = 0; i < n; i++)
    {
        int nge = -1;
        node *q = p;
        // look at the other n - 1 elements after p
        for (int k = 1; k < n; k++)
        {
            if (q == r)
                q = f;              // rear reached, wrap to front
            else
                q = q->next;

            if (q->data > p->data)
            {
                nge = q->data;
                break;
            }
        }
        cout << p->data << " -> " << nge << endl;
        p = p->next;
    }
}
```

7.4 Q5: sort (sortList, qsortList, csortList)

What it does: sorts the values of the list with **bubble sort**. Only the *data* is swapped. No node is moved, so top, f and r stay where they are and no reference parameters are needed.

Stack: sortList

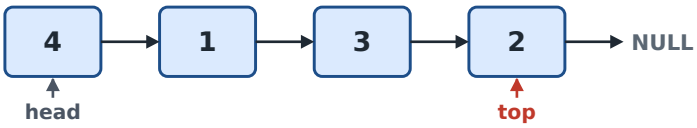
Parameter	Passed how	Why the function needs it
head, top	by value	The sort works on the used nodes from head to top. Nothing about the pointers changes, so both are by value.

1. Count the used nodes n (walk head to *top->next*). There is no stack count function, so the count is done here.
2. Outer loop i = 0 .. n - 2. Inner loop j = 0 .. n - 2 - i, with p starting at head.

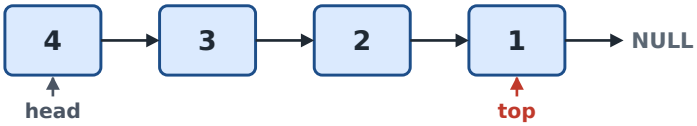
3. If *p->data* < *p->next->data* swap the two values. Then p = p->next.

The swap condition is '<'. That puts the **bigger values at the bottom** and the smaller values at the top. The menu prints from the top, so the stack appears in ascending order.

Before: bottom to top 4 1 3 2. Menu shows 2 3 1 4



After: bottom to top 4 3 2 1. Menu shows 1 2 3 4




```
void func::sortList(node *head, node *top)
{
    if (top == NULL)
        return;
    int n = 0;
    node *p = head;
    node *end = top->next;
    while (p != end)
    {
        n++;
        p = p->next;
    }
    for (int i = 0; i < n - 1; i++)
    {
        p = head;
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (p->data < p->next->data)
            {
                int t = p->data;
                p->data = p->next->data;
                p->next->data = t;
            }
            p = p->next;
        }
    }
}
```

Queue: qsortList, what is different
Parameters: *f* and *r* by value (no head).
Process: n comes from qcount. The condition is *p->data* > *p->next->data*, so the **smallest value ends at the front**. The queue's printing order is front to rear, so it is printed 1 2 3 4.

```
void func::qsortList(node *f, node *r)
{
    int n = qcount(f, r);
    for (int i = 0; i < n - 1; i++)
    {
        node *p = f;
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (p->data > p->next->data)
            {
                int t = p->data;
                p->data = p->next->data;
                p->next->data = t;
            }
            p = p->next;
        }
    }
}
```

Circular queue difference
csortList is the same as qsortList with n from ccount. No wrap is needed because the nodes from f to r are used in order.

7.5 Q6: palindrome (isPalindrome, qisPalindrome, cisPalindrome)

What it does: tells whether a word reads the same forwards and backwards. It uses the structure itself as the tool, with the **temporary list** (stmp, qtmp, ctmp) so the user's own list is not touched.

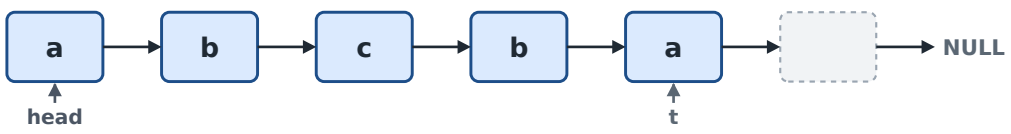
Stack: isPalindrome

Parameter	Passed how	Why the function needs it
s	by value (string)	The word to check. It is not a list, because the question is about a text, not about one of the user's lists.

There is no list parameter on purpose: the function uses its own local top (*node *t = NULL*) with the member list *stmp*. Because *t* is local and starts as NULL each time, the temporary list is always empty at the start. The menu refuses a word longer than 6 letters (it prints the overflow message), because the temporary list has only 6 nodes.

1. Push every character of s onto the temporary stack.
2. Pop the characters. The first pop gives the **last** letter, the second the second-last and so on, so popping reads the word **backwards**.
3. Compare the i-th popped letter with *s[i]*, which reads the word forwards. Count how many are equal.
4. If the count equals the length it is a palindrome (return 1), otherwise return 0.

Pushed a b c b a. Popping gives a b c b a again (reversed), compared with s = a b c b a



The nodes hold characters as numbers (their ASCII codes). The picture shows the letters for clarity.

```
int func::isPalindrome(string s)
{
    node *t = NULL;
    int len = s.length();
    for (int i = 0; i < len; i++)
        push(stmp, t, s[i]);

    int same = 0;
    for (int i = 0; i < len; i++)
    {
        if (pop(stmp, t) == s[i])
            same++;
    }
    if (same == len)
        return 1;
    return 0;
}
```

Queue: qisPalindrome, what is different
Temporary list: qtmp, with local front and rear (*tf, tr*).
Process: a queue returns values in the **same** order they went in, so it does not reverse anything by itself. To compare against the word backwards, the code reads the string from the other end: *for (i = len - 1; i >= 0; i--)* and compares with qpop.

```
int func::qisPalindrome(string s)
{
    node *tf = NULL, *tr = NULL;
    int len = s.length();
    for (int i = 0; i < len; i++)
        qpush(qtmp, tf, tr, s[i]);

    int same = 0;
    for (int i = len - 1; i >= 0; i--)
    {
        if (qpop(qtmp, tf, tr) == s[i])
            same++;
    }
    if (same == len)
        return 1;
    return 0;
}
```

Circular queue difference
cisPalindrome is the same as the queue version but with cpush and cpop and the circular temporary list ctmp.

7.6 Q7: copy (copyStack, copyQueue, copyCircular)

What it does: inserts every value of a list, in order, into another list. It is also the function Q1 uses to make its copy.

Stack: copyStack

Parameter	Passed how	Why the function needs it
head, top	by value	The source stack. Walk from head to top. Only read.
ch	by value	Head of the destination stack. push needs it for the first value.
ct	by reference	Top of the destination. It is set by the function, so the caller must see it. It is reset to NULL first so the copy starts empty.

The walk goes from the bottom to the top and calls the **base push**, so the copy has the same order and the same overflow behaviour.

```
void func::copyStack(node *head, node *top, node *ch, node *&ct)
{
    ct = NULL;
    if (top == NULL)
        return;
    node *p = head;
    node *end = top->next;
    while (p != end)
    {
        push(ch, ct, p->data);
        p = p->next;
    }
}
```

Queue: copyQueue, what is different
Parameters: the source is (*f, r*) and the destination is (*ch, cf, cr*): two pointers set by the function, both by reference.
Process: the walk goes from f to *r->next* and calls **qpush**.

```
void func::copyQueue(node *f, node *r, node *ch, node *&cf, node *&cr)
{
    cf = NULL;
    cr = NULL;
    if (f == NULL)
        return;
    node *p = f;
    node *end = r->next;
    while (p != end)
    {
        qpush(ch, cf, cr, p->data);
        p = p->next;
    }
}
```

Circular queue difference
copyCircular uses a **do...while** and **cpush**, for the same reason as cdisplay (a full ring).

```
void func::copyCircular(node *f, node *r, node *ch, node *&cf, node *&cr)
{
    cf = NULL;
    cr = NULL;
    if (f == NULL)
        return;
    node *p = f;
    node *end = r->next;
    do
    {
        cpush(ch, cf, cr, p->data);
        p = p->next;
    } while (p != end);
}
```

7.7 Q8: balanced brackets (isBalanced, qisBalanced, cisBalanced)

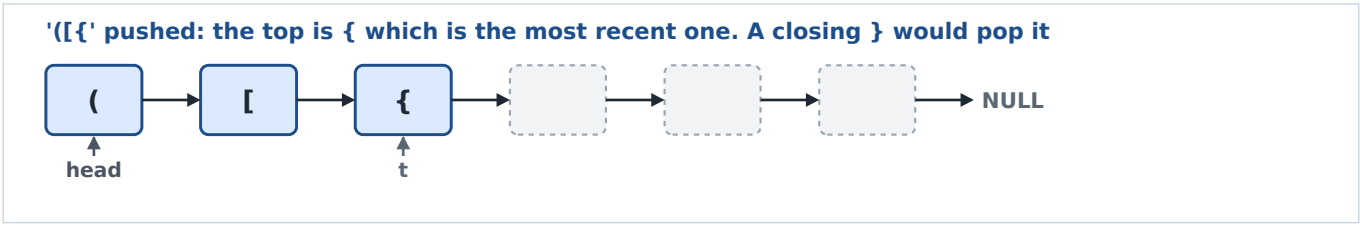
What it does: checks that every bracket ({ [is closed by the right bracket) }] in the right order. The rule: a closing bracket must match the **most recent** opening bracket that is still open.

Stack: isBalanced

Parameter	Passed how	Why the function needs it
s	by value (string)	The text of brackets to check. The function uses its own temporary stack (stmp with a local top).

1. For an opening bracket: **push** it.
2. For a closing bracket: if the stack is empty return 0 (nothing to match). Otherwise pop the top. It must be the matching opening bracket, or return 0.

3. At the end the stack must be empty. A leftover opening bracket means it is not balanced.



The most recent opening bracket is always at the top, so the stack is the natural tool for this check.

```
int func::isBalanced(string s)
{
    node *t = NULL;
    int len = s.length();
    for (int i = 0; i < len; i++)
    {
        char ch = s[i];
        if (ch == '(' || ch == '{' || ch == '[')
            push(stmp, t, ch);
        else if (ch == ')' || ch == '}' || ch == ']')
        {
            if (t == NULL)
                return 0;
            char open = pop(stmp, t);
            if (ch == ')' && open == '(')
                continue;
            if (ch == '}' && open == '{')
                continue;
            if (ch == ']' && open == '[')
                continue;
            return 0;
        }
    }
    if (t == NULL)
        return 1;
    return 0;
}
```

Queue: qisBalanced, what is different
Problem: the bracket to match is the **newest** one, but a queue only gives back its **oldest** (the front).
Solution: use **qpopLast** (section 6.2) which removes the rear, so the queue is made to behave like a stack. That is why qpopLast exists.
Parameters: the same as the stack version: the string only.

```
int func::qisBalanced(string s)
{
    node *tf = NULL, *tr = NULL;
    int len = s.length();
    for (int i = 0; i < len; i++)
    {
        char ch = s[i];
        if (ch == '(' || ch == '{' || ch == '[')
            qpush(qtmp, tf, tr, ch);
        else if (ch == ')' || ch == '}' || ch == ']')
        {
            if (tf == NULL)
                return 0;
            char open = qpopLast(tf, tr);
            if (ch == ')' && open == '(')
                continue;
            if (ch == '}' && open == '{')
                continue;
            if (ch == ']' && open == '[')
                continue;
            return 0;
        }
    }
    if (tf == NULL)
        return 1;
    return 0;
}
```

Circular queue difference
cisBalanced is the same with **cpush** and **cpopLast** on the circular temporary list.

7.8 Q9: remove duplicates (removeDuplicates, qremoveDuplicates, cremoveDuplicates)

What it does: removes repeated values so that each value appears once. It empties the list into a temporary list, keeping only values not seen yet, then moves them back.

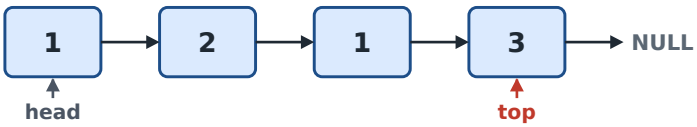
Stack: removeDuplicates

Parameter	Passed how	Why the function needs it
head	by value	pop needs it (prevNode), and the final push back needs it as the stack's own head.
top	by reference	The list is emptied and refilled, so top changes. The caller must see it.

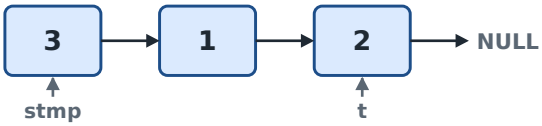
1. Pop everything from the stack. For each popped value x: if **search(stmp, t, x) == 0** (x is not in the temporary stack yet) push it onto the temporary stack.
2. Pop the temporary stack and push each value back into the original stack. Two reversals bring the order back to what it was.

Which duplicate is kept? Popping goes from the top, so the first copy found is the one nearest the top. The stack 3 1 2 1 (menu, top first) becomes **3 1 2**: the lower 1 is removed.

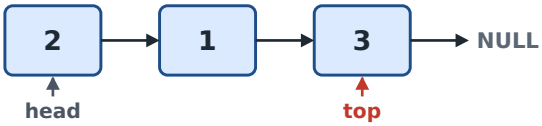
Original, bottom to top: 1 2 1 3 (menu shows 3 1 2 1)



Temporary stack after the loop: 3, 1, 2 pushed. The second 1 was skipped



After moving back: bottom to top 2 1 3 (menu shows 3 1 2)



```
void func::removeDuplicates(node *head, node *&top)
{
    node *t = NULL;
    // move unique values into the temporary stack
    while (top != NULL)
    {
        int x = pop(head, top);
        if (search(stmp, t, x) == 0)
            push(stmp, t, x);
    }
    // move them back
    while (t != NULL)
        push(head, top, pop(stmp, t));
}
```

Queue: qremoveDuplicates, what is different

Parameters: (*head, f, r*) with **head and f and r all by reference**. qpop recycles nodes and changes head, so qremoveDuplicates must pass its head to qpop by reference.

Process: qpop from the front, qsearch in the temporary queue, qpush into the temporary queue, then qpop it and qpush back.

Which duplicate is kept: a queue pops its oldest value first, so the **first** copy is kept and the list keeps its order: 1 2 1 3 becomes **1 2 3**. (No double reversal is needed, because a queue does not reverse.)

```
void func::qremoveDuplicates(node *&head, node *&f, node *&r)
{
    node *tf = NULL, *tr = NULL;
    // move unique values into the temporary queue
    while (f != NULL)
    {
        int x = qpop(head, f, r);
        if (qsearch(tf, tr, x) == 0)
            qpush(qtmp, tf, tr, x);
    }
    // move them back
    while (tf != NULL)
    {
        int x = qpop(qtmp, tf, tr);
        qpush(head, f, r, x);
    }
}
```

Circular queue: cremoveDuplicates, what is different

Parameters: *head* by value (cpop does not rotate nodes, so head never changes), *f* and *r* by reference.

Process: the same steps with cpop, csearch and cpush on the circular temporary list.

```
void func::cremoveDuplicates(node *head, node *&f, node *&r)
{
    node *tf = NULL, *tr = NULL;
    // move unique values into the temporary circular queue
    while (f != NULL)
    {
        int x = cpop(f, r);
        if (csearch(tf, tr, x) == 0)
            cpush(ctmp, tf, tr, x);
    }
    // move them back
    while (tf != NULL)
    {
        int x = cpop(tf, tr);
        cpush(head, f, r, x);
    }
}
```

7.9 Q10: intersection (intersection, qintersection, cintersection)

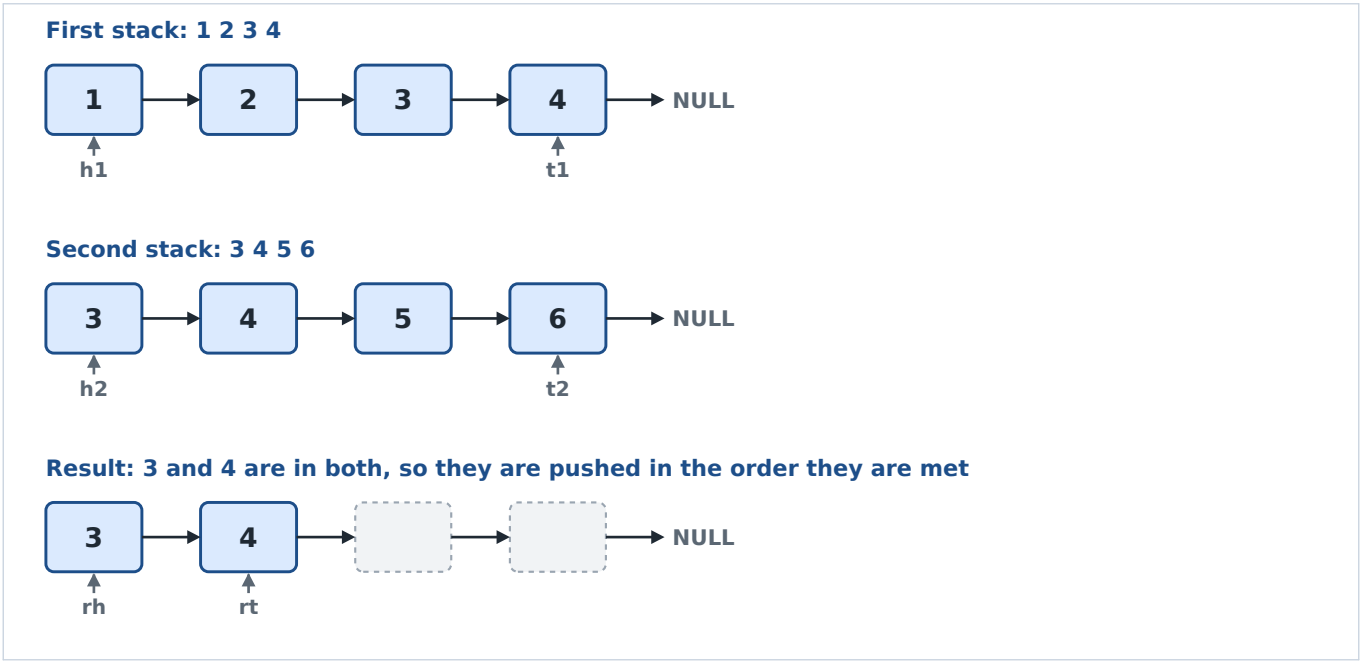
What it does: finds the values that appear in **both** of two lists and puts them into a third (result) list, once each.

Stack: intersection

Parameter	Passed how	Why the function needs it
h1, t1	by value	The first stack. We walk through its values, so both pointers are only read.
h2, t2	by value	The second stack. It is only searched, never changed.
rh	by value	Head of the result stack. push needs it for the first value.
rt	by reference	Top of the result stack. It is set by the function, so the caller must see it.

1. Set *rt* = *NULL* (the result starts empty). If either stack is empty, return: nothing can be common.
2. Walk p through the first stack from h1 to *t1->next*.
3. For each value: **search(h2, t2, value) == 1** (it is in the second stack) **and search(rh, rt, value) == 0** (not already in the result). If both are true, push it into the result.

The two conditions are joined with **&&**. When the first is false, the second is not even tested. The second condition is what prevents a value that appears twice in the first list from being added twice.




```
void func::intersection(node *h1, node *t1, node *h2, node *t2, node *rh, node *&rt)
{
    rt = NULL;
    // either stack empty, so nothing is common
    if (t1 == NULL || t2 == NULL)
        return;
    // walk the first stack from bottom to top
    node *p = h1;
    node *end = t1->next;
    while (p != end)
    {
        // push if it is in the second stack and not already in the result
        if (search(h2, t2, p->data) == 1 && search(rh, rt, p->data) == 0)
            push(rh, rt, p->data);
        p = p->next;
    }
}
```

Queue: qintersection, what is different

Parameters: every list is described by two pointers (f, r) with no head: the sources (*f1, r1*) and (*f2, r2*) by value; the result (*rh, rf, rr*) with *rf* and *rr* by reference.
Process: the same two-condition test with qsearch and qpush, walking f1 to *r1->next*.

```
void func::qintersection(node *f1, node *r1, node *f2, node *r2, node *rh, node *&rf, node *&rr)
{
    rf = NULL;
    rr = NULL;
    // either queue empty, so nothing is common
    if (f1 == NULL || f2 == NULL)
        return;
    // walk the first queue from front to rear
    node *p = f1;
    node *end = r1->next;
    while (p != end)
    {
        // insert if it is in the second queue and not already in the result
        if (qsearch(f2, r2, p->data) == 1 && qsearch(rf, rr, p->data) == 0)
            qpush(rh, rf, rr, p->data);
        p = p->next;
    }
}
```

Circular queue: cintersection, what is different

Parameters: the same as the queue.
Process: a ring has no NULL end, so the first list is walked a **counted** number of times: *n = ccount(f1, r1)* and a for loop. The tests use csearch and the insert is cpush.

```
void func::cintersection(node *f1, node *r1, node *f2, node *r2, node *rh, node *&rf, node *&rr)
{
    rf = NULL;
    rr = NULL;

    // either circular queue empty, so nothing is common
    if (f1 == NULL || f2 == NULL)
        return;

    // walk the first circular queue, counted (a ring has no NULL end)
    int n = ccount(f1, r1);
    node *p = f1;
    for (int i = 0; i < n; i++)
    {
        // insert if it is in the second one and not already in the result
        if (csearch(f2, r2, p->data) == 1 && csearch(rf, rr, p->data) == 0)
            cpush(rh, rf, rr, p->data);
        p = p->next;
    }
}
```

8. The menus: how the functions are connected

The functions in parts 3 to 7 never ask the user anything. The **menu functions** do the asking, pick the list to work on, check the input and call those functions. There are five: **main** (in main.cpp), **menu**, **stackMenu**, **queueMenu** and **circularMenu**.

8.1 Start-up: main and menu

1. **main.cpp** creates one object, *func f;*, and calls *f.menu()*.
2. **menu()** builds one stack, one queue and one circular queue with createList (list number 1 of each kind, so snum, qnum and cnum are 1). It also builds the three temporary lists stmp, qtmp and ctmp (the circular one with circular = 1).
3. It then shows the main menu (1 Stack, 2 Queue, 3 Circular Queue, 0 Exit) and calls stackMenu, queueMenu or circularMenu. Each of those loops until the user chooses 0 (Back).

```
void func::menu()
{
    // start with one of each kind, plus the temporary lists
    createList(shead[0], 0);
    snum = 1;
    createList(qhead[0], 0);
    qnum = 1;
    createList(chead[0], 1);    // circular: last node points back to first
    cnum = 1;

    createList(stmp, 0);
    createList(qtmp, 0);
    createList(ctmp, 1);

    int c = 0;
    do
    {
        cout << "\n===== MAIN MENU =====" << endl;
        cout << "1. Stack" << endl;
        cout << "2. Queue" << endl;
        cout << "3. Circular Queue" << endl;
        cout << "0. Exit" << endl;
        cout << "Choice: ";
        cin >> c;
        if (c == 1)
            stackMenu();
        else if (c == 2)
            queueMenu();
        else if (c == 3)
            circularMenu();
    } while (c > 0);
}
```

8.2 Working on the selected list

The first lines inside the stack menu loop bind two names to the selected stack:

```
        // h and t are the head and top of the selected stack
        node *&h = shead[scur];
        node *&t = stop[scur];
```

h and **t** are **references** (*node* *&) to the array slots *shead[scur]* and *stop[scur]*. They are not copies. Calling *push(h, t, x)* therefore moves the real *stop[scur]*. This is why the base **push**, **pop** and **display** work on any stack and there is no *push1* or *push2*. Changing which stack is selected means changing *scur*, and h and t re-bind on the next loop.

	Stack menu	Queue menu	Circular menu
Bound names	h, t	h, f, r	h, f, r
Arrays used	shead, stop	qhead, qfront, qrear	chead, cfront, crear
Counters	snum, scur	qnum, qcur	cnum, ccur
Base functions	push, pop, display	qpush, qpop, qdisplay	cpush, cpop, cdisplay

8.3 Create, select and show all

- **Create a new list:** checks there is room (*snum >= MAXS*), builds a new list in slot *snum* with createList, sets its pointers to NULL, makes it the selected list (*scur = snum*) and then increases snum.
- **Select a list:** asks for a number from 1 to snum, rejects numbers outside that range, and sets *scur = n - 1* (1-based for the user, 0-based for the array).
- **Show all:** loops from 0 to snum - 1 and calls the display function for each list.

```
case 13:
    if (snum >= MAXS)
    {
        cout << "No room for another stack" << endl;
        break;
    }
    createList(shead[snum], 0);
    stop[snum] = NULL;
    scur = snum;                // the new stack becomes the selected one
    snum++;
    cout << "Stack " << scur + 1 << " created and selected" << endl;
    break;

case 14:
    cout << "Stack number (1 to " << snum << "): ";
    cin >> n;
    if (n < 1 || n > snum)
        cout << "Wrong stack number" << endl;
    else
    {
        scur = n - 1;
        cout << "Stack " << n << " selected: ";
        display(shead[scur], stop[scur]);
    }
    break;
```

> **Queue and circular menus**
> The queue and circular menus have the same three choices (numbers 15, 16 and 17) with the three arrays of their kind.

8.4 Menu choices that create new lists (Q1, Q2, Q7, Q10)

The results of these questions are real lists, not temporary output. Each choice follows the same pattern:

1. **Check the input:** the list must not be empty, and there must be room in the array (*snum + 2 > MAXS* for two new lists, *snum + 1 > MAXS* for one).
2. **Create the new lists** in the next free slots of the arrays (*snum*, *snum + 1*). Q1 builds them inside the split function, Q2, Q7 and Q10 call createList in the menu.
3. **Call the question function** with the selected list and the new slots.
4. **Increase the counter** (snum += 2 or snum++), print the old and the new lists and tell the user which numbers to select.

Q1 (split):

```
int a = snum, b = snum + 1;
if (splitStack(h, t, shead[a], stop[a], shead[b], stop[b]) == 0)
{
    cout << "Stack has fewer than 2 nodes, it cannot be split" << endl;
    break;
}
snum += 2;
```

Q2 (even/odd) in the stack menu:

```
// two new stacks at the end of the array: e for even, o for odd
int e = snum, o = snum + 1;
createList(shead[e], 0);
stop[e] = NULL;
createList(shead[o], 0);
stop[o] = NULL;
snum += 2;

evenOdd(h, t, shead[e], stop[e], shead[o], stop[o]);
```

> **Why the original list is never overwritten**
> Older versions put these results into shared global lists, which destroyed them the next time. Writing into a **new** slot each time means that the user can run Q2 or Q7 again and again and still have every earlier list.

8.5 Q10 in the menu: four checks

Q10 asks for **two** lists to intersect, so the menu must check the input before it calls **intersection**:

Check	Code idea	Why
1	a or b is outside 1 .. snum	The number must name a list that exists, otherwise *shead[a]* would be a NULL or invalid slot.
2	a == b	Intersecting a list with itself gives the same list. The question needs two different lists.
3	stop[a] == NULL or stop[b] == NULL	An empty list has no values, so nothing can be common.
4	snum + 1 > MAXS	There must be room for the result list.

After the checks the numbers are converted to 0-based (*a--; b--;*), a new result list is created, **intersection** is called, and both inputs and the result are printed.

```
case 12:
{
    int a = 0, b = 0;
    cout << "First stack (1 to " << snum << "): ";
    cin >> a;
    cout << "Second stack (1 to " << snum << "): ";
    cin >> b;

    // Check 1: both numbers must be stacks that exist
    if (a < 1 || a > snum || b < 1 || b > snum)
    {
        cout << "Wrong stack number" << endl;
        break;
    }
    // Check 2: the two stacks must be different
    if (a == b)
    {
        cout << "Choose two different stacks" << endl;
        break;
    }
    a--;                        // user numbers start at 1, the array at 0
    b--;
    // Check 3: neither stack may be empty
    if (stop[a] == NULL)
    {
        cout << "Stack " << a + 1 << " is empty, push some values first" << endl;
        break;
```

```
        }
        if (stop[b] == NULL)
        {
            cout << "Stack " << b + 1 << " is empty, push some values first" << endl;
            break;
        }
        // Check 4: room for the result stack
        if (snum + 1 > MAXS)
        {
            cout << "No room for another stack" << endl;
            break;
        }

        int res = snum;             // the result goes in a new stack
        createList(shead[res], 0);
        stop[res] = NULL;
        snum++;

        intersection(shead[a], stop[a], shead[b], stop[b], shead[res], stop[res]);
```

8.6 Word questions (Q6 and Q8)

Palindrome and brackets read a **word**, not a list. The menu checks *s.length() > maxsize* first and prints the overflow message of that structure, because the temporary list has only 6 nodes and could not hold more letters. Only if the word fits does it call isPalindrome or isBalanced and print the result.

```
    case 8:
        cout << "Enter string: ";
        cin >> s;
        if ((int)s.length() > maxsize)
            cout << "Stack Overflow: string is longer than " << maxsize << " characters" << endl;
        else if (isPalindrome(s) == 1)
            cout << "Palindrome" << endl;
        else
            cout << "Not Palindrome" << endl;
```

9. Comparison of the three structures

	Stack	Queue	Circular Queue
Working ends	One: the top	Two: front (remove) and rear (insert)	Two, and both can wrap around the ring
Last node's next	NULL	NULL	head
Insert function	push(head, top&, x)	qpush(head, f&, r&, x)	cpush(head, f&, r&, x)
Insert moves	top = top->next	r = r->next	r = r->next, or back to head (wrap)
Overflow when	top->next == NULL	r->next == NULL	f == head and r->next == head (simple), or r->next == f (circular)
Remove function	pop(head, top&)	qpop(head&, f&, r&)	cpop(f&, r&)
Remove moves	top moves **backwards** with prevNode (slow, searches from head)	f moves forward; the removed node is recycled after r; head = f	f moves forward; the wrap is automatic, nothing is recycled
Underflow when	top == NULL	f == NULL	f == NULL
Becomes empty by	top = NULL	f = r = NULL	f = r = NULL (next insert starts at head)
Display order	top to bottom (newest first)	front to rear (oldest first)	front to rear, through the wrap, with do...while
Needs prevNode for	pop, display, nextGreater	qreverse, qpopLast	creverse, cpopLast
Loop end	p != top->next	p != r->next	do...while (p != r->next), or a counted loop
Q1 split halves	values reversed in half 2 so both tops meet at the cut	front to rear 1 2 3 and 4 5 6	two rings, each closed with its own link
Q6 palindrome	pop reverses, compare forwards	read the string backwards	read the string backwards
Q8 brackets	pop	qpopLast (rear)	cpopLast (rear)
Q9 duplicates	keeps the copy nearest the top; double reversal	keeps the first copy	keeps the first copy

The one idea behind all the differences

A **stack** has one end, so it needs only one pointer, but because the nodes only point forward it must search from head to go back one step. A **queue** has two ends, so it needs two pointers, and each end moves forward, which needs no searching. A **circular queue** joins the last node to the first so that rear can reuse the space freed at the front, and this makes 'full' harder to detect, which is why it needs the four checks.

10. Likely viva questions

Q1. Why is the list fixed-size?
createList builds all 6 nodes at the start. Insert and remove only move pointers. Overflow means there is no free node left.

Q2. How do you detect overflow in each structure?
Stack: *top->next == NULL*. Queue: *r->next == NULL*. Circular: *f == head && r->next == head* for the simple case, or *r->next == f* after wrapping.

Q3. Why does pop on a stack need head?
The nodes have only a next pointer. To move top one step down, prevNode searches from head for the node whose next is top.

Q4. Why does qpop take head by reference?
qpop moves the old first node to just after the rear and sets head = f. The caller's head must change too.

Q5. Why does the circular queue not need head in cpop?
No node is moved. f = f->next wraps by itself because the last node points to head.

Q6. Why do cdisplay, ccount and csearch use do...while?
When the ring is full, r->next equals f. A normal while loop would stop at once. do...while runs the first step before testing.

Q7. Explain the four checks of cpush.
1 empty: f and r go to head. 2 f == head and r->next == head: full as a simple queue. 3 r->next == f: circular full. 4 otherwise move r, and if it is on the last node it goes back to head.

Q8. How does Q1 split work?
Copy the base list, find the k-th node with k = (m + 1) / 2, cut with mid->next = NULL (ring: mid->next = first node and last->next = h2). The halves overflow at k and m - k nodes.

Q9. Why is the second stack reversed after the split?
So its top sits next to the cut (value 4). Stack 1 shows 3 2 1 and stack 2 shows 4 5 6.

Q10. Why does Q10 check that the two lists are different and not empty?
Intersecting a list with itself returns the same list, and an empty list has nothing in common. The checks also stop bad list numbers from being used.

Q11. How can the same push work for any of 100 stacks?
The menu binds references h and t to *shead[scur]* and *stop[scur]*. push always gets the selected stack's real pointers.

Q12. Why does the bracket check for a queue use qpopLast?
The bracket to match is the newest one, which is at the rear. A queue gives back the oldest, so we remove from the rear instead.